The Journal of Systems and Software 230 (2025) 112555

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

# The DevSafeOps dilemma: A systematic literature review on rapidity in safe autonomous driving development and operation☆

Ali Nouri [a,c,*], Beatriz Cabrero-Daniel [a,b], Fredrik Törner [c], Christian Berger [a,b]

[a] Chalmers University of Technology, Department of Computer Science, Gothenburg, Sweden
[b] University of Gothenburg, Department of Computer Science, Gothenburg, Sweden
[c] Volvo Cars, Gothenburg, Sweden

ARTICLE INFO

*Keywords:*
Continuous development
Safety-related function
Autonomous driving
Safety of the intended function (SOTIF)
DevOps
DevSafeOps

ABSTRACT

Developing autonomous driving (AD) systems is challenging due to the complexity of the systems and the need to assure their safe and reliable operation. The widely adopted approach of DevOps seems promising to support the continuous technological progress in AI and the demand for fast reaction to incidents, which necessitate continuous development, deployment, and monitoring. We present a systematic literature review meant to identify, analyse, and synthesise a broad range of existing literature related to usage of DevOps in autonomous driving development. Our results provide a structured overview of challenges and solutions, arising from applying DevOps to safety-related AI-enabled functions. Our results indicate that there are still several open topics to be addressed to enable safe DevOps for the development of safe AD.

## 1. Introduction

Artificial Intelligence (AI) has seen widespread adoption in various fields over the past decade, including the automotive industry. Autonomous Driving (AD) and Advanced Driver Assistance Systems (ADAS) are currently two prominent scenarios to apply AI. Ensuring the robustness and safety of such AI-based systems in a dynamic environment is a complex and multifaceted process that requires continuous monitoring and fast yet systematic reactions to the possible identified hazards.

AD development started back in the 1980s (Gudla et al., 2022) and there have been local experiments; however recent loss events highlight its ongoing safety challenges. For instance, in a recent mishap involving a robotaxi, a pedestrian was severely injured (Lawyer, 2024; Koopman, 2024). According to the investigation report (Lawyer, 2024), the cause was neither hardware nor software failure. The AD perception detected both the pedestrian and the adjacent vehicle. Weak recognition and response to nearby incidents, along with an inaccurate post-crash world model, are some of the technical issues and challenges highlighted by Koopman (Koopman, 2024). Moreover, a human safety driver is also considered a potential solution for handling these unforeseen complex scenarios (Koopman, 2024). This incident highlight the need for novel approaches in the design of ADS that enable the system to adapt its behaviour to similar unforeseen, complex, and yet numerous events. Releasing the system without sufficient confidence and necessary fallback strategies not only leads to safety risks but also poses delays in deployment or termination of the project. For instance, the aforementioned example led to the immediate revocation of ADS deployment and testing permits by the California Department of Motor Vehicles (DMV).[1]

The innovation rapidity in the automotive industry necessitates companies to adopt continuous development and integration approaches to remain competitive. Continuous approaches, such as DevOps, aim at function growth and refinement to lead to better customer experience after each design iteration (Google, 2024b). Hence, DevOps has the potential to enable continuous loops of monitoring and software adaptation for AD. It facilitates the expansion of the Operational Design Domain (ODD). Additionally, DevOps is crucial for maintaining the safety of the system against detected anomalies, and improve the adaptation speed to context/environmental/regulation updates (Zeller et al., 2024; Weiss et al., 2024). Hence, the safety community aims to integrate some aspects of DevOps into standards such as ISO/PAS 8800 (Safety and Artificial Intelligence) and ISO/TS 5083 (Safety for Automated Driving Systems).

However, the safety process required by automotive standards such as ISO 26262 ISO 26262:2018, 2018 or ISO 21448 (2022), and regulations such as UNECE R157 (ALKS) (UNECE, 2021) can make it

☆ Editor: Dr. Dario Di Nucci.

* Corresponding author at: Chalmers University of Technology, Department of Computer Science, Gothenburg, Sweden.
*E-mail address:* ali.nouri@volvocars.com (A. Nouri).

[1] DMV STATEMENT ON CRUISE LLC SUSPENSION, accessed February 28, 2025, https://www.dmv.ca.gov/portal/news-and-media/dmv-statement-on-cruise-llc-suspension/.

https://doi.org/10.1016/j.jss.2025.112555
Received 7 September 2024; Received in revised form 15 April 2025; Accepted 26 June 2025
Available online 11 July 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

challenging to utilise full capacity of DevOps. These standards prescribe engineering rigour for increasing integrity and reducing safety risks, which are essential in safety-related domains. On the other hand, the fundamental concept of DevOps is to emphasise speed and rapid updates. This can be perceived as a dilemma as speed and safety seem to not be able to be combined at a glance. As accident reports show, this dilemma has, in some cases, been resolved in favour of speed and innovation rather than safety. One notable example occurred in the avionics industry, where prioritising speed and competitiveness over thorough safety processes during the modification of existing aircraft models not only led to fatalities but also resulted in the worldwide grounding of that model (DeFazio and Larsen, 2020).

However, DevOps does not dictate an absolute level of speed; rather, it is a mindset focused on continuous improvement and adaptability in relative terms, compared to earlier practices within an organisation or even external organisations. Hence, DevOps applied in safety related domains implies that the integrity and safety is viewed as prerequisites, but within these boundaries the DevOp principles are applied to its full extent.

To the best of the author's knowledge, there is no systematic attempt to synthesise the research on this challenge targeting in particular the automotive domain. This study aims at reviewing the literature on safety-related DevOps challenges, which are considered to be applicable for AD, and explores proposed solutions and mitigation strategies by answering the following research questions:

- **RQ1.** What challenges are identified in literature when applying DevOps to safety-related AD functions?
- **RQ2.** What solutions are proposed in literature for these challenges while fostering rapidity and safety?
- **RQ3.** What challenges for DevOps in safety-related AD applications still remain open in literature?

We followed a systematic approach, based on the work by Kitchenham and Charters (Kitchenham and Charters, 2007), to synthesise a broad range of literature on DevOps engineering process in safe AD development. The results map current challenges of DevOps for safety-related AI-enabled functions (RQ1) to suggested state-of-the-art solutions (RQ2). The analysis also reveals several opportunities for research to better address the identified challenges (RQ3).

The rest of this paper is organised as follows. Background and related work on DevOps and safety-related AI-enabled functions are presented in Sections 2 and 3. The research design and implementation are described in Section 4. The results of the study are presented and discussed in Sections 5 and 6 concludes the paper.

## 2. Background

### 2.1. Safety - Definitions, regulations and standards landscape

To assure and argue the safety of ADS, the 'safety criteria' shall first be defined, both qualitatively and quantitatively. Several stakeholders are involved in defining the criteria, including Consumers, industries involved in the ADS development, and legislators.

United Nations Economic Commission for Europe (UNECE) published UN Regulation No. 157 (UNECE, 2021), which defines qualitative and quantitative criterion to ensure the safety of Automated Lane Keeping Systems (ALKS). As a qualitative requirements, UNECE R157 recommends compliance with ISO 26262 (ISO 26262:2018, 2018), ISO 21448 (ISO 21448, 2022), and ISO 21434 (ISO 21434, 2021) by requiring a competent auditor and assessor in these standards.

ISO/TR 4804[2] (ISO/TR 4804:2020, 2020) defines it as the Absence of Unreasonable Risk (AUR). ISO 26262, ISO 21448, and ISO 8800 refine the definition (i.e., AUR), making it more specific based on the root causes they target:

ISO 26262: "AUR due to hazards caused by **malfunctioning behaviour of E/E systems**"

ISO 21448: "AUR due to hazards resulting from **functional insufficiencies of the intended functionality** or **its implementation**."

ISO 8800: "AUR due to **errors of the AI system**."

Positive Risk Balance (PRB), which requires ADSs to "cause fewer crashes on average compared to those made by drivers," can be employed as a quantitative safety criterion (ISO/TR 4804:2020, 2020) and acceptance criterion in ISO 21448 (ISO 21448, 2022). However, this raises questions about "how many less?" and "which driver?" (Koopman and Widen, 2023). As highlighted by Koopman and Widen, both AUR and PRB concepts fall short for ADS, since they fail to account for scenarios requiring moral and contextual understanding and might lead to risk redistribution to other road users (Koopman and Widen, 2024).

ISO 26262 (ISO 26262:2018, 2018) (functional safety, or so-called FuSa) provide process requirements to avoid or mitigate systematic failures in hardware and software, as well as random hardware faults, during the design, implementation, verification, validation, and field monitoring phase. The resulting work-products from these activities are then used as evidences in a safety case to argue the achievement of functional safety for the item.

As a complementary standard to ISO 26262, ISO 21448 (Safety of Intended Functionality, or SOTIF) (ISO 21448, 2022) contains requirements and recommendations for avoiding hazardous events caused by functional insufficiencies, incorrect or inadequate Human–Machine Interface (HMI), and insufficiencies in Artificial Intelligence (AI)-based algorithms. SOTIF requires the definition of acceptance criteria, which can be both qualitative and quantitative (e.g., one event per X km), as the first level of safety requirements. The fulfilment of each acceptance criterion is then argued based on evidence from analysis and testing for each validation target (e.g., no hazardous behaviour during X hours of testing) (ISO 21448, 2022). If any of the acceptance criteria are not met, a functional modification (i.e., changes in design and specification) is needed.

ISO 4804 (ISO/TR 4804:2020, 2020) and the upcoming ISO 5083 are ADS-specific standards that discuss additional process aspects of ADS in relation to ISO 26262 and ISO 21448. Additionally, they provide technical and architectural recommendations for ADS. UL 4600 is another ADS-specific standard that focuses on system-level aspects (Koopman, 2023). There are also general safety standards, such as IEC 61508 (International Electrotechnical Commission (IEC), 2010), which serve as the foundation for other domain-specific safety standards such as ISO 26262 and ISO 21448.

Employing AI technology in automotive software is not specific to ADS. However, relying on it without human supervision for safety-related tasks in a complex environment presents a new challenge for the automotive industry, which ISO 8800[3] aims to address.

National Highway Traffic Safety Administration (NHTSA) (U.S. Department of Transportation NHTSA, 2017) encourages companies to submit a publicly available Voluntary Safety Self-Assessment (VSSA) report, which outlines how they address safety concerns, including system safety, ODD, validation methods, and data recording. While this is not mandatory, the VSSA report aims to promote transparency and build public trust. As of today, 28 companies have a link to their VSSA or their safety page describing their key safety design elements, which are analysed in our study from an industrial perspective.

[2] Road vehicles - Safety and cyber-security for automated driving systems - Design, verification and validation.

[3] ISO/CD PAS 8800, accessed May 20, 2024, https://www.iso.org/standard/83303.html.

### 2.2. Abstraction levels

Automotive system engineering requires a well-established modular and multi-abstraction architecture and requirements engineering process. Each abstraction level consists of multiple modules, each of which is abstracted by removing or merging unnecessary characteristics of the elements in the module (Committee, 2022). Traceability between the elements and requirements at the most abstract level and the most granular level is essential to enable impact analysis and maintainability of the system after modifications during the life cycle of the system. The recommended abstraction levels are as follows, but are not limited to these, as they depend on the scope and complexity of the product:

**Concept** phase starts with a short description of the item, describing its functionality and ODD, known as the 'Item Definition' (ISO 26262:2018, 2018). ODD is a set of predefined restrictions on the function due to its limitations to ensure safety (Stettinger et al., 2024). It also includes the item's boundary diagram, which represents its interaction with the environment, operator, and other agents. 'Hazard Analysis & Risk Assessment' (so called HARA in the context of ISO 26262 ISO 26262:2018, 2018) and Hazard Identification & Evaluation (in the context of ISO 21448, 2022) are then employed to identify all hazardous events that result from the occurrence of malfunctions or functional insufficiencies in a specific scenario. A safety goal (ISO 26262 ISO 26262:2018, 2018) or an acceptance criterion ISO 21448 (2022) is specified for the safety-related hazardous events to avoid or mitigate.

**Function** level is the next abstraction level, which includes the functional architecture, inductive or deductive safety analysis methods and functional safety requirements all gathered in a functional safety concept (ISO 26262:2018, 2018) and a SOTIF safety concept (ISO 21448, 2022). Ideally, the functional/SOTIF safety requirements are technology-agnostic.

**System** level is the first level where technical aspects of a function are designed. A technical safety concept is the container for system-level (technical) safety requirements resulting from design choices based on the safety analysis results.

**Software** and hardware levels are the last abstraction levels. Software safety requirements are derived from system level technical safety requirements employing safety analysis and the safety solutions are designed in the software architectural design. Depending on the complexity of the system, these levels can be further broken down into more levels due to the complexity of the product or other constraints.

## 3. Related work

A limited body of academic literature addresses some of the challenges of integrating DevOps practices in the development of safety-related software. Moreover, there is a growing set of literature that recognises the difficulties of integrating AI into such systems. These challenges are illuminated by grey literature, multi-vocal literature reviews, or interviews such as Nouri et al. (2022).

DevOps couples Continuous Development (Dev) and Operations (Ops) (Munk and Schweizer, 2022). Primary needs include monitoring, diagnostics, and event recording, satisfied by triggers and data loggers (Munk and Schweizer, 2022; Fayollas et al., 2020). Using DevOps in an automotive engineering context can enable rapid communication towards original equipment manufacturers (OEMs) to allow new software releases, enabled by over-the-air updates (Fayollas et al., 2020; Czarnecki, 2019; Kugele and Broy, 2022; Nouri et al., 2022), which may be even more rapid by 5G and 6G (Kugele and Broy, 2022). Flexible system architectures such as central computing units allow for safe integration of new software without compromising the safety of current mature software (Fayollas et al., 2020). The UP2DATE (Agirre et al., 2020) European project aimed to propose new software update architectures for mixed-criticality systems.

Machine Learning (ML) is a crucial technology in AD, especially for supporting perception tasks. However, unbounded complexity of the environment and distributional shift (Borg et al., 2019) requires continuous monitoring, data gathering and improvement during operation (i.e., MLOps) (Siddique, 2020; Cârlan et al., 2022; ISO 21448, 2022; Google, 2024a; John et al., 2023). Hence, successful ML development requires an iterative, close, and rapid connection between operation and development (Borg, 2021).

The SafeOps approach by Munk and Schweizer focuses on ensuring safety in iterative development processes (Munk and Schweizer, 2022). Similarly, the SafetyOps framework by Siddique addresses challenges of integrating safety and DevOps and emphasises the need to adapt DevOps practices to meet the unique needs for safety (Siddique, 2020). Considering the provided terms and definitions, we propose DevSafeOps to refer to the integration of safety and DevOps. The absence of "Dev" in the other two terms (i.e., SafeOps Munk and Schweizer, 2022; Fayollas et al., 2020, SafetyOps Siddique, 2020) make them less suitable compared to ours, where the concept of "safe by design" (Galbas et al., 2024) plays an important rule. Moreover, the term SafeOps is used in other contexts, such as safe military operations (Kott III et al., 2011).

To the best of the authors' knowledge, none of the aforementioned studies systematically focuses on and reviews DevOps challenges for AI-enabled safety-related automotive functions such as AD with the goal of mapping them to state-of-the-art solution suggestions. However, the literature does report useful knowledge for the field on specific topics like verification and validation (V&V) (Borg et al., 2019), agile methodologies in safety (Myklebusta et al., 2022), or cyber-security (Sánchez-Gordón and Colomo-Palacios, 2020). Thus, the present paper aims to fill this gap, providing a comprehensive view on rapidity in safe AD DevOps, as described below.

## 4. Method: a systematic literature review

The present study uses a Systematic Literature Review (SLR) in order to explore the current landscape of rapidity for developing safe AD DevOps. Prior to the start of the study, we queried Google Scholar to retrieve papers containing the keywords "DevOps," "Safety," and "Systematic Literature Review." This pilot search demonstrated that there is no available SLR on safety of DevOps, since the retrieved papers focus instead on other aspects such as quality (Céspedes et al., 2020) or cyber-security (Sánchez-Gordón and Colomo-Palacios, 2020).

After this preliminary study, we designed a methodology to gather, analyse and synthesise relevant literature on the topic at hand as described below. Given the high industrial relevance of this endeavour, we opted to include grey literature and non-peer reviewed articles, which enriches the results.

### 4.1. Data sources and search strategy

To begin this process, we conducted preliminary searches to identify existing systematic reviews and to assess the volume of potentially relevant studies. With this analysis, we identified which keywords are used within each relevant research topic, and what are boundaries of the existing literature. This information was used to design a query to retrieve existing literature on safety and rapid software development. For the selection of primary studies, we queried four literature databases (ACM Digital Library, IEEE Xplore, and Scopus) and Google Scholar, that indexes also pre-print servers such as arXiv, using the following query:

```
safety AND (26262 OR 21448 OR SOTIF)
AND (DevOps OR MLOps) AND
(((autonomous OR automated OR driverless) AND (vehicle OR drive
OR driving OR car)) OR self − driving)
```

The result of this query was then assessed by the first and second authors to identify the primary studies to be evaluated and synthesised in this study, as reported in Fig. 1.

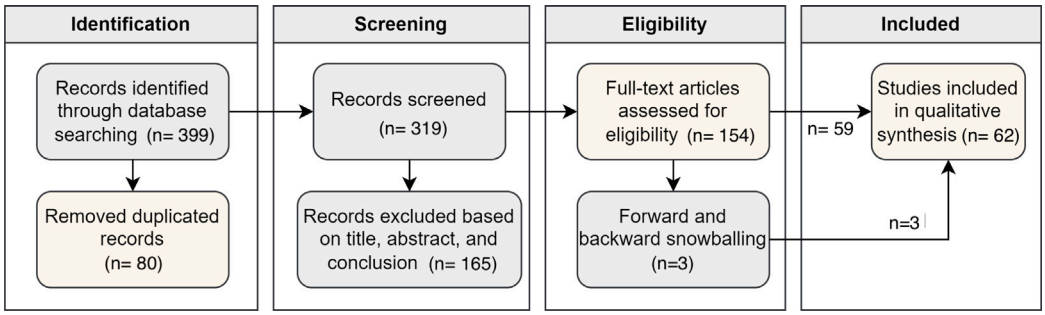


**Fig. 1.** Flow diagram depicting the application of inclusion and exclusion criteria.

### 4.2. Study selection

Selection criteria were defined to dictate whether a study would be included or excluded from the set of primary studies to analyse. The design of these criteria was based on preliminary exploratory efforts. For instance, given that older studies seemed to discuss fundamental yet still relevant challenges, no study was excluded because of its publication date. As a result, the earliest identified publication is from 2015. As a result, retrieved papers are included if they meet the following inclusion criteria:

**Inc.1** Written in English.
**Inc.2** Presenting challenges, problems, and confounding factors in applying DevOps (or MLOps) to safety-related functions in automotive, or discussing practices, guidelines, lessons learned, methods, algorithms, approaches, or tools to address the identified challenges.

The "exclusion criteria" are defiend as following:

**Exc.1** Does not discuss about safety of DevOps or MLOps in the automotive domain.
**Exc.2** Does not report on any challenges or solutions.
**Exc.3** Is not written in English.

The first author acted as the main assessor evaluating each prospective primary study, while the second author independently evaluated all included records, and explored a sample of the excluded studies (random selection and most cited ones) to validate the exclusion rational. There were no discrepancies between the evaluations made by the first and second authors. Fig. 1 summarises the selection process at each stage.

### 4.3. Data extraction and synthesis

After removing duplicate records from the results of the database search, we screened the records based on title, abstract, and conclusion, using the aforementioned criteria (screening phase). Then, in the eligibility phase, the full text was analysed and relevant records were selected. If additional records were identified through snowballing, they were also included (see Supplementary Materials[4]).

The goal was to extract information related to applying DevOps in safety-related aspects of AD or applicable to AD. Retrieved information includes the first author, publication year, publication venue, research type, and industrial sector. Moreover, data to address each research question was retrieved, i.e., challenges identified (RQ1), proposed software technologies, and recommendations to address specific issues (RQ2), and open challenges and future work directions (RQ3).

Then, thematic analysis was performed on the primary studies. First, papers were read to form initial ideas for analysis and generating inductive codes from lists of challenges, practices, and factors. These codes were combined into potential themes, later refined by merging, balancing, or dropping codes. By doing so, we grouped the challenges and solutions based on safety activities and relevant categories. Clear and concise terms for each challenge and solution were defined to present the results in Section 5.

In addition, we aimed at aggregating the extracted information to derive clusters of challenges and solutions, our work also gets a notion of a mapping study. In that regard, we aimed at maintaining the integrity of the original information by only linking challenges to solutions if they are mapped in the original authors' publication. We acknowledge, though, that some solutions could be applied to multiple challenges and some challenges may overlap with other groups. For that reason, we clearly indicate and justify solutions that are not mapped to a specific challenge in literature. Furthermore, we provide a mapping aimed to address **RQ3** that combines existing studies and serves as a basis to discuss the interrelation and inter-dependency of the reported challenges.

### 4.4. Complimentary studies: the industrial perspective

In order to highlight the importance of industrial contributions to this field of study, we mark any cited material involving industrial authors with $[X]^I$. Moreover, a complementary set of reports, produced by industrial parties, was analysed and included alongside the retrieved papers, as "Industrial Context": We analysed all Voluntary Safety Self-Assessment (VSSA) reports from companies involved in ADS development, which are published on the NHTSA website.[5] Snowballing was also performed from these reports to identify other relevant reports or publications.

### 4.5. Threats to validity

Different types of information sources have been used, due to the nature of the research in the scope of the automotive industry. Therefore, the clustering could be sensitive to the way that the review has been conducted. For instance, safety case, assessment, certification, and impact analysis challenges are related to more granular safety activities due to their interconnection in different abstraction levels, which could lead to other possible clustering patterns. To mitigate this, we followed the structuring pattern used in ISO 26262, which forms the foundation of automotive safety. Moreover, this clustering was done for presentation and communication purposes, and it does not influence the data extraction process or the conclusions drawn. Lastly, since the

[4] Supplementary Material regarding included and excluded data, and the rationale, https://zenodo.org/records/14946487.

[5] NHTSA Voluntary Safety Self-Assessment, accessed February 28, 2025, https://www.nhtsa.gov/automated-driving-systems/voluntary-safety-self-assessment.

clustering was applied only after data synthesis, it does not introduce bias or affect the internal validity of the study.

While solutions from other domains can be used in automotive, we focused on prescribed approaches for automotive, as otherwise, such potential approaches require to be validated if applicable to such standards, which is outside the scope of this literature review. Moreover, the automotive domain has its own constraints and characteristics such as complexity, competitiveness, speed, and distributed development way of working, which might be different in other industries.

This work might be missing or excluding some relevant studies on safety-related challenges in continuous development approaches. To mitigate this, we used different digital libraries with a query that was designed iteratively based on the independent initial findings of the authors with the goal to complement each other. Moreover, snowballing was used (see Fig. 1) to screen other related studies, strictly following the pre-defined review protocol in Section 4, defining the inclusion and exclusion rationales. Moreover, additional sources, mainly grey literature and AD developer's VSSA reports, have been added to cover potentially existing gaps in academic literature. The second author performed a cross-check on all the included papers and a randomly selected set of the excluded studies (25% over the final set, corresponding to one every four of the papers listed in the Supplementary Materials) to reassess whether they had to be included or not in each stage of the screening process.

## 5. Results

This section describes the synthesis of reviewed papers, aimed to answer our research questions. To this end, challenges ($CH_n$) and proposed solutions ($Sol_{n,m}$) are listed here, grouped based on the results of our thematic analysis. We integrated the required safety activities into our interpretation of the DevOps infinity loop, which is shown in Fig. 2, and then mapped all identified challenges to the activities. To structure the findings from the reviewed papers, they were clustered into 11 challenges, each aligned with key activities or phases defined in ISO 26262. For each challenge we first present our own analysis regarding the relevance of proposed solutions and other possible approaches are discussed at the end of each challenge in "*Discussion*". Finally, for each challenge, the findings and analysis results from VSSA reports are presented in "*Industrial Context*". Our concluding interpretations and reflections are discussed in Section 6.

### CH1: Safety and software workflow separation

According to Siddique in his 2020 position paper "SafetyOps", safety activities are traditionally expected to be performed independently from design activities (Siddique, 2020)$^I$. According to Fayollas et al. such practice may also result in conflicts between the development team and safety experts (Fayollas et al., 2020)$^I$. As a consequence, safety activities have been primarily managed by safety experts, with little engagement from the development team. This is specifically true for Agile teams (Warg et al., 2019)$^I$, which results in misalignment and delays in releases. The safety experts referred to here are those engaged in producing safety artifacts and designs, while confirmation review tasks (e.g., assessment) necessitate separation and independence.

*Sol1.1:* Hanssen et al. proposed "SafeScrum" for Agile Development of safety-related Software (Hanssen et al., 2018) $^I$, introducing additional roles with respect to traditional Scrum that are required to meet the requirements of IEC 61508 (International Electrotechnical Commission (IEC), 2010). A "Quality Assurer" is a Scrum team member with relevant safety knowledge who ensures that software quality assurance tasks, the safety plan, and the safety validation plan are completed (Hanssen et al., 2018)$^I$, (Myklebust et al., 2020)$^I$.

*Discussion:* Potential root cause of this challenge is the lack of safety knowledge within the development team in some companies. Following the SafeScrum (Hanssen et al., 2018)$^I$ proposal can help to address this issue by including at least one safety expert in the team, ensuring the quality of safety activities, and guiding the team in performing the tasks. The daily stand-up meetings facilitate collaboration between developers and safety experts, enabling them to update each other and correct misalignment faster. This, together with knowledge sharing (T-shaped skills) and training sessions, could increase the safety skills of the team. Independent tester, "reliability, availability, maintainability and safety (RAMS)" engineer, and independent safety assessor are external to the team, with I2 and I3 degrees of independence (Hanssen et al., 2018)$^I$. The independent safety assessor, who is not part of the product development or the same organisation (i.e., I3 degree of independence), receives evidence of compliance and ensures that the requirements from the standard are fulfilled.

*Industrial Context:* According to IKE's Safety Report (IKE, 2025), it is the responsibility of the systems engineering team to ensure the safety of their work and the potential implications of their engineering decisions. Their responsibilities also include peer-reviewing software before commits and performing safety analyses, such as STPA.

### CH2: Requirement updates

Requirements engineering for safety-related applications typically consists of multiple abstraction levels, starting with safety goals in the context of ISO 26262 or validation targets and acceptance criteria in ISO 21448 and then, are broken down into more granular ones (Nouri et al., 2022)$^I$. The DevOps approach starts with a Minimum Viable Product (MVP) that is iteratively improved by adding features (Czarnecki, 2018), and then new features are added to the product to improve the user experience continuously. Then it is necessary to update/remove legacy requirements and to add new ones in each iteration. This made traditional requirements engineering inadequate in capturing benefits of rapid DevOps cycles (Bosch et al., 2018). Bosch et al. report that requirements are typically agreed before the implementation phase, and sudden changes are avoided because they are estimated to be costly (Bosch et al., 2018). Incorporating multiple confirmation measures such as verification reviews, confirmation reviews, or assessments in requirements engineering result in even more needed effort and potential delays. According to Habibullah et al. the inclusion of ML elements makes it challenging to employ requirement traceability to trace the source of issues. Therefore, it is necessary to consider requirement traceability for models and datasets as well (Habibullah et al., 2024)$^I$. For instance, in AD, it is also important to link safety goals to scenarios.

*Sol2.1:* AI- and data-driven development are proposed as two potential alternatives to traditional requirements engineering (Bosch et al., 2018; Habibullah et al., 2024)$^I$. *AI-driven development* is widely used in development of perception software of AD, such as object detection, by using ML techniques. ML is trained on large data-sets of pictures or point clouds of objects instead of relying on detailed requirements about shape and appearance of an object. *Data-driven development* enables the team to use data in addition to requirements to develop software, and then use metrics to evaluate the performance of the developed software (Bosch et al., 2018; Kugele and Broy, 2022; Hartmannsgruber et al., 2024)$^I$. In both cases, confidence in system performance is initially low due to insufficient data. Thus, by introducing limitations and a conservative approach, safety is maintained. Later, supported by data, unnecessary constraints can be identified and removed (Warg et al., 2024)$^I$, (Warg et al., 2019)$^I$. Continuous data collection and monitoring of the system and the environment through event detectors and monitoring tools help to identify emerging edge cases (Czarnecki, 2018).

*Discussion:* The use of data as an input for ML-based software development is discussed in ISO 21448 (2022) and upcoming standards such as ISO/TS 5083 and ISO/PAS 8800. Nevertheless, it is not yet explored as a replacement for safety-related requirements at any abstraction level. Moreover, although ML-based software is utilised

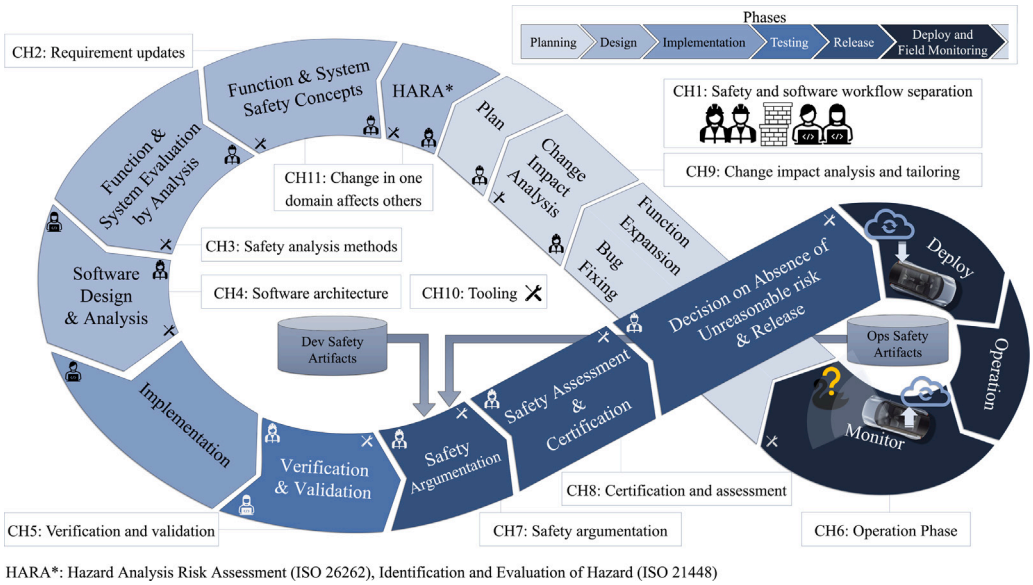


**Fig. 2.** The figure outlines the safety activities that are necessary to be carried out in DevSafeOps cycles, along with the corresponding challenges identified in this study. The asymmetrical shape of the loop illustrates the significant number of activities required on the development side, as compared to the operation phase. The loop starts from planning in the first iteration, and in the successive iterations, it begins with Function expansion or bug fixing. The artefacts created during the Development and Operation phases of DevSafeOps are continuously collected to form the safety argumentation. The intensity of the colour represents different phases in the DevSafeOps loop.

in perception systems, where data is employed as a requirement for software behaviour, there are still requirements on the utilised data and ML technology. These requirements are in turn used in the safety V&V process of the entire perception system and its components.

*Industrial Context:* According to VSSA of Torc (Torc-Robotics, 2025), ODD evolution over time is decomposed into testing ODD (T-ODD) and ODD expansions (eODDs). T-ODD refers to the ODD that the current software is capable of handling and is being tested for, while eODD represents the ODD planned for the next phase of software development and expansion. Finally, after multiple evolutions, the deployment ODD is formed, indicating the ODD of the deployed software. However, the report does not describe the impact of this iterative ODD expansion on the evolution of safety requirements or other artifacts.

## CH3: Safety analysis methods

"Deductive safety analysis" (Top-Down) methods are employed to identify the potential causes of the higher level safety requirement violation. For instance, Fault Tree Analysis (FTA) can be employed to identify the potential root causes for requirements in higher abstraction level (e.g., safety goals violation) in the current abstraction level (e.g., functional modules).

"Inductive safety analysis" (Bottom-Up) methods are used to identify the effects of a specific failure mode or performance limitation on higher abstraction levels. If the effect is related to a violation of safety requirements at higher levels, then the failure mode or performance limitation shall be avoided or mitigated. For instance, Failure Mode and Effect Analysis (FMEA) approach starts from the failure modes of each module at the current level (e.g., function level) and identifies the effects at higher levels (e.g., violating safety goal).

Analysis methods such as "System Theoretic Process Analysis" (STPA), "Fault Tree Analysis" (FTA), and "Failure Mode and Effect Analysis" (FMEA) are performed manually and separately without considering other methods (Siddique, 2020)$^{I}$ (Nouri et al., 2023). This can lead to inconsistency between the results of different analysis methods. Moreover, safety analysis shall be updated whenever a change occurs in the system or new evidence and events are collected from the field; otherwise, the results may become outdated or incorrect (Thomas and Groth, 2023). Reviewing and updating of them is a manual and time consuming task. This may not only cause delays in releasing new software or system (Siddique, 2020)$^{I}$, but also raises the risk of late reactions to field data, which might lead to an accident. Additionally, the elements in each method (e.g., failure mode, architectural elements, effects) should be traceable across different abstraction levels, enabling the tracing of causes to the most granular level (Nouri et al., 2023)$^{I}$.

*Sol3.1:* Integrating all safety analysis methods into a single framework and connecting the data to the relevant parameters enable automatic updating of said parameters with each data update (Siddique, 2020)$^{I}$. For instance, in this way the frequency of failures in FMEA can be linked directly to field data (Siddique, 2020)$^{I}$ and being updated automatically instead of traditional manual failure calculation using an outdated failure catalogue. This might require adaptations of methods, such as the proposed tailoring of STPA in Nouri et al. (2023)$^{I}$, to enable modularisation and multi-abstraction levels while maintaining traceability between elements down to the most granular level. Using unified causal model (i.e., cause and effect relationships) for safety analysis can be a potential solution. Maier et al. proposed a framework for integrating causal models into DevOps, called CausalOps (Maier et al., 2024). Similarly, Hybrid Causal Logic (HCL) methodology (Thomas and Groth, 2023) proposes a data-driven and dynamic safety analysis. In this approach, safety analysis is integrated with Event Sequence Diagrams (ESD) and Bayesian Networks, enabling continuous and dynamic improvements. HCL leverages Bayesian Networks to update safety analysis results based on new field data. Moreover, ESD integration ensures that all possible failure paths and scenarios are considered in the analysis.

*Sol3.2:* Zeller et al. (2024)$^{I}$ posed the question of whether it is possible to automate hazard identification and risk assessment, a challenge that has been addressed and implemented by Nouri et al. (2024b)$^{I}$. As safety requirements are in natural language, hence Natural Language Processing (NLP) can be a potential solution, an idea that has been technically implemented. The impressive performance of Large Language Models (LLMs) in natural language understanding and text generation has led to their use in requirements engineering and safety analysis (Nouri et al., 2024b)$^{I}$ (Nouri et al., 2024a)$^{I}$. Nouri et al. (2024b)$^{I}$ designed a pipeline consisting of LLM agents (GPT-4 with specific prompt templates), each specialised in a subtask of HARA, communicating with one another to deliver a HARA for any function, including safety goals. These models can also be used for data collection and preparation in natural language from various sources, such as

police accident data containing challenging scenarios specific to each country. For instance, Bäumler et al. used BERT to classify German police accident data into 30 different turning accident types (Bäumler and Prokop, 2024).

*Discussion:* Data-driven development, discussed in *Sol2.1*, is a prerequisite for *Sol3.1*. This emphasises the importance of integrating data in every aspect of the development process such as requirements engineering and analysis activities. As mentioned in *Sol3.2*, the use of NLP techniques is not limited to safety analysis and can be expanded to other activities documented in natural language. However, the results of these studies must be validated rigorously with special attention. For instance, as highlighted by Nouri et al. (2024b)$^{I}$, testing LLMs against existing open-source baselines only assesses their performance on data they might have been trained on and does not validate their performance for new input.

*Industrial Context:* Safety analysis is a "living" activity throughout the development cycle and should be updated based on field data (IKE, 2025). For instance, near-loss incidents should be analysed to update the system or software and to prevent future failures (IKE, 2025). Moreover, in analysis methods such as STPA, failure modes related to the software update process (e.g., incomplete updates or incorrect versions) should also be analysed, and corresponding mitigation strategies can be developed (IKE, 2025).

## CH4: Software architecture

AD requires a flexible and scalable architecture, which enables continuous updates (Lee et al., 2024). Therefore, its architecture is not limited to onboard computing; it also extends to cloud-based services that communicate with onboard systems through real-time communication (Lee et al., 2024). Amalfitano et al. emphasise on the critical role of software architecture in complex safety-related applications such as AD (Amalfitano et al., 2024), a point also reported by Kugele et al. in "Architecture as a Backbone for Safe DevOps in Automotive Systems" (Kugele and Broy, 2022).

Since architecture does not evolve as quickly as software itself, it can become a bottleneck for functional growth, highlighting the need for continuous architecture along with evolving documentation and reasoning (Bosch et al., 2022). Outdated architectural documentation in each development loop and the inability to assess cohesion and coupling between components are some of the reported issues (Amalfitano et al., 2024). Additionally, due to centralised architecture and the possibility of both, safety-related and non-safety-related components coexisting on the same system, there is a risk of failure propagation, which could lead to the violation of safety requirements (Kugele and Broy, 2022).

*Sol4.1:* Ensuring Freedom From Interference (FFI) is essential for avoiding cascading failures in periodical updates (Kugele and Broy, 2022). To fulfil FFI and enable continuous development and integration after each innovation delivery, Kugele et al. propose hypervisors for partitioning between safety-related and non-safety-related software that is executed on the same processor (Kugele and Broy, 2022). Kugele et al. also introduced *containerisation* in their 2018 paper, "Data-Centric Communication and Containerisation for Future Automotive Software Architectures" (Kugele et al., 2018). According to Lee et al. container technology on top of the operating system enables a division between safety and non-safety applications (Lee et al., 2024).

*Discussion:* As a result of continuous function improvement or ODD expansion, new decomposition patterns need to be applied, which impose new requirements on the software architecture. For instance, in the case of Automotive Safety Integrity Level (ASIL) decomposition, the decomposed components must not have any common cause failures, which imposes requirements on shared resources. Hence, all these restrictions need to be considered, which may challenge flexibility. Moreover, as the architecture extends to cloud-based services, which are not directly addressed in the mentioned safety standards, it becomes challenging to consider them in safety activities using conventional standards.

*Industrial Context:* Nvidia highlights the importance of scalability in the SW architecture to enable functional growth, and support multiple levels of integrity and autonomy, from Level 2 to Level 5, through integration of additional hardware and software (NVIDIA, 2025). This also applies to different levels of safety, as OEMs may adjust their safety requirements (e.g., redundancy or diversity) in response to ODD or functionality expansions.

## CH5: Verification and validation

Testing is essential for the certification and safety of autonomous systems; however, the infinite state-space makes it impractical to test all corner cases in a real-world setting (Siddique, 2020)$^{I}$, a challenge known as the "long tail" of events (Hartmannsgruber et al., 2024)$^{I}$. These events are critical as they are rare and they were most likely not part of the training data, leading to weaknesses in the software's ability to handle these events (John et al., 2023). Hence, it is essential to plan and design a granular verification and validation strategy for each technology used in AD, depending on their requirements, limitations, and weaknesses. For instance, ML-based software testing involves three main phases, each with its own purpose: during model training, offline pre-deployment, and online post-deployment testing (Borg, 2021). Adhering to such a thorough and lengthy testing process is challenging due to frequent changes in relatively short iterations (Stapp et al., 2024). Moreover, modifying such a complex software requires extensive validation, significantly increasing development expenses (John et al., 2023). Ideally, the verification and validation aspects of safety argumentation should allow for minor system updates without requiring the repetition of major parts of the tests (Hoss et al., 2022). However, this cannot be achieved without suitable methods, strategies, and technical approaches.

*Sol5.1:* Simulations can cover a wider range of scenarios by using catalogues and randomising them (Siddique, 2020)$^{I}$. It can be even used for Simulation-Aided Hazard and Risk Analysis (SAHARA), which assists a safety engineer in identifying of where to perform a Hazard Analysis Risk Assessment (Meyers et al., 2019). However, three key aspects must be considered: Ensuring the relevance of the scenario to the ODD, maintaining traceability between the scenario and safety requirements, and calculating test coverage (Siddique, 2020)$^{I}$. Software-in-the-Loop (SiL) is also proposed as a virtual environment for continuously testing new software by utilising the virtual version of the hardware and other software components (Raghupatruni et al., 2021)$^{I}$, (Haas et al., 2024)$^{I}$. Model-based simulation (Blanco et al., 2023; Mörtstrand, 2023; Tomar, 2024)$^{I}$, including software and relevant electrical and mechanical aspects (Haas et al., 2024)$^{I}$, can be employed for high-level testing and capturing design faults in the early phases of development. Simulation models can also be used to generate synthetic raw data and its ground truth (e.g., object list) for testing perception algorithms without the need for physical sensors (Hoss et al., 2022) for diverse and unseen scenarios (Schuster et al., 2023)$^{I}$ and data-driven validation (Hartmannsgruber et al., 2024)$^{I}$, (Habibullah et al., 2024)$^{I}$. The synthetic and real-world data are then combined to avoid deviating from reality, addressing the so-called "Reality Gap" (Schuster et al., 2023)$^{I}$.

*Sol5.2:* Before entering operative mode, where the software sends unsupervised commands to the actuators, the so called shadow mode approach allows the system to be validated by perceiving and planning without being executed (Zeller et al., 2024)$^{I}$. Shadow mode tests new software versions under real operational conditions, measuring the readiness of it for real world implementation (Blanco et al., 2023)$^{I}$, (Hartmannsgruber et al., 2024)$^{I}$, (Fayollas et al., 2020)$^{I}$. This concept can be used to provide evidence for the safety case and achieve continuous learning and data collection in each version (Johansson

and Koopman, 2022)$^I$, (Czarnecki, 2019). For instance, if discrepancies in detected objects occur between two decomposed paths during operation, the pertinent data is gathered to examine the underlying cause (Johansson and Koopman, 2022)$^I$.

*Sol5.3:* Giaimo et al. (2019), Borg (2021) and Fayollas et al. (2020)$^I$ discuss canary and A/B testing as methods for evaluating software during the operational phase, which is known as *continuous experimentation*. Canary testing introduces new software to a random, small user sample to identify errors prior to full deployment, while A/B testing employs specific user selection criteria.

*Sol5.4:* Test automation (Stapp et al., 2024; Mörtstrand, 2023) improves efficiency and speed in each iteration, especially for tasks that are frequent, such as ML updates, which are continuously refined by collected data. As ML is the foundation of perception in AD, it is crucial to verify and validate its performance after each training or fine-tuning cycle in MLOps, or when the model is exposed to a new environment. Employing techniques such as "Gradient-weighted Class Activation Mapping (Grad-CAM)" (Borg et al., 2021) improves the explainability of the model and enables automation. Grad-CAM provides a heatmap that shows which part of the image the model focuses on to decide on the classification of an object, which can be automated by comparing the highlighted area with the expected one (Borg et al., 2021).

*Discussion:* Although simulation (Sol5.1) has benefits, including testing harmful scenarios without causing any hazard, correlating between simulation and reality is a challenging task (Czarnecki, 2019; Habibullah et al., 2024)$^I$. Moreover, increasing the precision of simulations increases the computational cost of each run, making it impractical for achieving full coverage, especially for complex systems such as AD in multi-agent and complex environments. This highlights the need for smart selection of use cases of simulation and the necessity of bringing real world solutions such as shadow mode (Sol5.2) or continuous experimentation (Sol5.3) for safe use cases. Test automation (Sol5.3) is beneficial as it reduces the risk of human errors due to its consistency and repeatability; however, it is not a replacement for "human critical thinking" (Stapp et al., 2024).

*Industrial Context:* To achieve rapid continuous integration, as highlighted in the Kodiak VSSA report (Kodiak, 2025), the latest build of each week is tested against the current stable software, and if the results show significant safety improvements, it is selected as the new stable version. According to the Mercedes-Benz's VSSA report (Mercedes-Benz, 2025), software components are tested using a well-defined test plan in various environments, such as Hardware-in-the-Loop (HIL) and Vehicle-in-the-Loop (VIL) tests conducted on proving grounds, before being tested on public roads. According to the AutoX Team (AutoX, 2025), efficient validation tools such as simulation, HIL, and VIL enabled them to update the software within two days. Minimising dependency on public road testing can accelerate the software update process (IKE, 2025). Since real-world testing has limitations, simulation enhances completeness by covering a wider range of scenarios, in which AD is tested (Mercedes-Benz, 2025). Hence, after a software update (e.g., for handling complex scenarios), testing in a simulation environment is conducted to ensure that the software still meets the requirements and that no significant regression has occurred (AVSC-D-02-2024, 2024). For instance, Waymo's team, in multiple reports and publications (Webb et al., 2020; Schwall et al., 2020; Waymo, 2025), describes the use of simulation to evaluate AD's crash avoidance capability. As another example, according to Aurora's VSSA report (Aurora, 2025a), their software was tested for more than two million left turns in simulation before performing one in the real world. Rapid software development and bug fixing, without introducing any safety risks, are among the benefits of simulation (Aurora, 2025a). Moreover, as testing all permutations of elements in a scenario is not feasible in the real world, simulation is a viable alternative (Aurora, 2025a). According to WAAbi's VSSA report (Waabi, 2025), a high-fidelity, closed-loop, end-to-end simulator (called Waabi World) is used for faster and more efficient testing rather than real-world testing. In some cases, simulation is used to examine the performance of the AD software in an alternative scenario rather than relying solely on collected data (Schwall et al., 2020). For instance, if a test vehicle operator assesses a risk of collision and intervenes, then simulation can be used to examine the performance of the software without operator interference (Schwall et al., 2020). According to Waymo (Webb et al., 2020), there is a need to rely on both simulation and real-world testing, as some tests are not possible in the real world and others are not possible in simulation. Moreover, virtual test scenarios originate from data collected on public roads or closed courses; however, sometimes they are generated from scratch solely for the simulation environment (Webb et al., 2020). Another crucial source for scenario generation is by sourcing from crash databases (Webb et al., 2020). Although crashes rarely occur, continuous improvement in the AD system's reactions during these scenarios may lead to either no collisions or a reduction in the severity of these events (Webb et al., 2020). Closed-course testing is also employed to validate the assumptions used in the development of simulation environments and to assess their accuracy (Webb et al., 2020). For instance, the braking profile in simulation should replicate that of the actual vehicle (Webb et al., 2020). As safety rating organisations such as New Car Assessment Programs (NCAPs) focus on current products, it is also important to continuously monitor the development of test scenarios and metrics proposed and considered by them (Webb et al., 2020). The use of a combination of simulation and on-road testing after each software update is also described in other reports (TUSIMPLE, 2025; Volkswagen, 2025; ZOOX, 2025; Apple, 2025; a2z, 2025; Ford, 2025).

### CH6: Operation phase

Autonomous systems operating in complex and unpredictable scenarios may encounter conditions that were not previously anticipated or even considered. This would lead to known unknowns (roughly predicted), unknown unknowns (completely missed) and difficulties in accurately quantifying the associated risks especially during design phase, according to Schleiss et al. (2022). Moreover, model drift is considered a potential threat during the operational phase, highlighting the need to identify it and, if necessary, retrain and update the models (John et al., 2023; Brajovic et al., 2023)$^I$. This is why the continuous monitoring of released functions is required not only by automotive standards and regulations but also by broader frameworks such as the EU AI Act. Article 9 of the AI Act mandates the implementation of a risk management system for continuous risk assessment and evaluation during operation (Brajovic et al., 2023)$^I$. Additionally, an iterative process for evaluating and improving mitigation strategies is required to ensure that AI-enabled systems operate safely; otherwise, the system must be disabled (i.e., fail-safe) (Brajovic et al., 2023)$^I$.

*Sol6.1:* Runtime monitoring dynamically verifies the function against safety requirements and triggers relevant mechanisms in case of violations (Furrer, 2022). The runtime monitor receives the necessary internal and external information and checks the system's behaviour against rules, contracts, or policies, which are often specified formally (Furrer, 2022). Gautham et al. implemented a model that enhances the hazard causation model of STPA to monitor the operation phase and detect hazards at run-time (Gautham et al., 2022). The dynamic risk assessment framework is claimed to improve safety assurance by monitoring, evaluating and responding to risks in real time. This framework, together with Employing Micro-Operational Design Domains ($\mu$ ODDs), prevents having worst-case assumptions during the design phase (Schleiss et al., 2022). ML techniques, such as Out-of-Distribution (OOD) detection, can also be employed to identify inputs that ML models have not encountered before (Hartmannsgruber et al., 2024)$^I$. Additionally, f-divergence can be employed to measure the difference between the data distribution during the operational phase and the training data used in the development phase, helping to quantify

data drift and detect significant deviations that may impact model performance (Brajovic et al., 2023)$^{I}$. OOD detection and f-divergence approaches are used not only to trigger relevant runtime risk mitigation strategies but also to collect and store data for model improvement.

*Sol6.2:* According to regulations, it is mandatory to equip AD-capable vehicles by "Data Storage System for Automated Driving (DSSAD)" (UNECE, 2021) to record and store critical events (e.g., detected collision, system failure). An automated system for rapidly identifying and reporting incidents, including near misses when human intervention prevents accidents, is proposed as a solution in Schleiss et al. (2022), UNECE (2021). Automation can encompass elements such as verifying assumptions and adjusting values, logging and reporting, narrowing down root cause search space, and distributing updates (Schleiss et al., 2022). For instance, utilising concepts such as safety cage, the DNN itself can employ novelty detection to continuously monitor the operational environment and report when uncertainty exceeds a certain limit (Borg et al., 2019)$^{I}$. The collected data will not only be used for offline development but can also be utilised for runtime learning and adaptation (Perez-Cerrolaza et al., 2024)$^{I}$. However, this concept, if applied without any constraints (e.g., limited actuation, limited adaptation), is not currently covered by existing standards (Perez-Cerrolaza et al., 2024)$^{I}$.

*Discussion:* Dynamic and Run-time Risk Assessment framework (Sol6.1) would try to prevent incidents or hazards to occur. If any incidents missed to be prevented, then smart triggers detect, record, and report (Sol6.2) them to the OEM, to prevent them and avoid potential accidents. However, run-time monitoring imposes extra demands on hardware (e.g., processing and memory overhead), which must be balanced against hardware limitations (Heyn et al., 2024) to avoid the risk of hardware resource shortages. ML performance assessment techniques, such as O$\underline{\text{O}}$D, can also be used to identify technological weaknesses of the components in the system and, if persistent, add new restrictions to the O$\underline{\text{D}}$D until those weaknesses are resolved.

*Industrial Context:* As NHTSA asks all companies to report on their "Data Recording" for analysing failures, malfunctions, or system degradation in the event of a crash (Torc-Robotics, 2025), the recorded data is used for event reconstruction and the evaluation of design and software performance, leading to continuous software improvement (GM, 2025). This is especially crucial for perception and localisation, which are responsible for reliable object and event detection as vital elements in ADS safety (Mercedes-Benz, 2025). As challenging environmental situations or data mismatches between different sensors might lead to failures in perception or localisation, these events are collected, analysed, and used for continuous improvement (Mercedes-Benz, 2025).

## CH7: Safety argumentation

A *safety case* demonstrates achievement of functional safety by providing a structured argument, supported by evidences derived from the prescribed activities in the product safety life-cycle. According to Myklebust et al. (2020)$^{I}$, it is called "safety case" since it must argue that the system is safe, similar to presenting a case in a court of law.

Safety cases of AD depend on factors such environmental conditions, which may shift during time, and system aspects, which would change on every update. This raises the need for continuously updating the safety case (Siddique, 2020)$^{I}$, otherwise it will be outdated and not valid (Schleiss et al., 2022). Another reason for continuous update is Continues Integration and Deployment (CI/CD) (Johansson and Sivencrona, 2021)$^{I}$, which can be due to bug fixing (Munk and Schweizer, 2022)$^{I}$, reconfiguration (Schleiss et al., 2022) or changes in the system design (Siddique, 2020)$^{I}$. Since safety is an overall system property, it is essential not only to prove the safety of the updated element but also to ensure that the safety of the entire system is maintained (Hake et al., 2024)$^{I}$.

The size and complexity of the system and its assurance case (e.g., safety case), as well as its change management, are the most significant problems raised by assurance case practitioners (Wardziński and Jarzebowicz, 2023). According to Munk and Schweiser, a safety case update for a bug fix may take up to two weeks (Munk and Schweizer, 2022)$^{I}$. This would take even longer for adding a feature or expanding the ODD, which would delay the feature update. For instance, specifying ODD requires particular attention, as the safety case is based on the assumptions derived from ODD (Weiss et al., 2024)$^{I}$; hence, its expansion would affect those assumptions and require updates to the safety argumentation.

*Sol7.1:* Incremental safety assurance (Hoss et al., 2022), agile safety case (Myklebust et al., 2020)$^{I}$, continuous safety case development (Johansson and Koopman, 2022)$^{I}$, (Wardziński and Jarzebowicz, 2023), (Siddique, 2020)$^{I}$ and the progressive release of a safety case by releasing each work product can prevent the need for a big bang approach, and leads to progressive functional safety assessments as recommended by ISO 26262 (Schleiss et al., 2022). The planning for a safety case should begin at the start of the development to identify which evidence is needed and how it can be generated during the development (Myklebust et al., 2020)$^{I}$. A well-connected framework between all components of a safety case, such as collected data, design and V&V artefacts, is a prerequisite for a machine-readable, hierarchical, and continuous safety case (Warg et al., 2024)$^{I}$, (Siddique, 2020)$^{I}$. Additionally, this approach leads to a forward-looking safety case as proposed by Warg et al. (2019)$^{I}$. This involves starting the safety case at the same time as development, allowing for the evaluation of safety arguments and strategies before implementation.

*Sol7.2:* The presence of uncertainty during the operational phase of the system in complex and unpredictable environments necessitates dynamic safety assurance (Sivakumar, 2024). Dynamic safety case and run-time monitoring is an approach used to evaluate the level of confidence or uncertainty of a safety case (Schleiss et al., 2022; Sivakumar, 2024). The avionics industry employs this approach that starts with an assurance architecture, which presents the risk reduction due to function and components. Then, the assurance rational itself is developed and finally, the assurance confidence is quantified. Using this, the safety of the system is assured during run-time (Schleiss et al., 2022).

*Sol7.3:* Modularisation of a safety case, which suggests that the safety case of the components can be developed separately as safety case fragments, is an effective approach to handle complexity (Wardziński and Jarzebowicz, 2023; Warg et al., 2019)$^{I}$. The safety case fragments are referenced by the overall safety case of the item to provide further details. This approach is recommended also by ISO 26262 for distributed development (Nouri et al., 2023)$^{I}$, (Warg et al., 2019)$^{I}$. This would reduce the needed re-certification effort of modules in case of reuse of them in the same context (Wardziński and Jarzebowicz, 2023; Warg et al., 2019)$^{I}$. Wardzinski and Jarzebowicz proposed the System Assurance Reference Model (SARM) (Wardziński and Jarzebowicz, 2023), a framework designed to facilitate assurance case development and management. This model allows for the decomposition of the assurance case and the automation of certain aspects of it.

*Sol7.4:* "Safety Element out of Context" (SEooC) (ISO 26262:2018, 2018) enables suppliers to begin the component development in parallel with the OEM, meaning at the same time or even sooner. This approach prevents development delays caused by missing inputs from an OEM by assuming the needs of an OEM, which will be validated when such inputs are available. This approach allows the supplier to deliver solutions promptly once the OEM identifies a need (Johansson and Sivencrona, 2021)$^{I}$. Although there is a requirement to validate the supplier's assumptions and a risk of mismatches that may necessitate changes, both parties can always determine the most efficient adjustment in either the OEM's or supplier's context, minimising costs and delays.

*Sol7.5:* "Proven in use" and "history in service" argument can be applied to components that were not developed based on standards but

have been used previously. It relies on a monitoring system or component behaviour during operation, determining satisfactory confidence levels based on incident-free performance over time (Fayollas et al., 2020)$^I$.

*Sol7.6:* Quantitative or qualitative confidence estimation is useful to demonstrate the level of trust that can be placed on the safety case, key for continuously assuring safety (Schleiss et al., 2022; Warg et al., 2024)$^I$. Schleiss et al. gathered qualitative or quantitative approaches for estimating confidence (Schleiss et al., 2022). For instance, The "Assurance Claim Points" approach can be used to demonstrate that the system is safe, and a contrapositive approach to prove that the system is unsafe. Safety Performance Indicators (SPIs) are quantitative metrics that use evidence and threshold comparison to make claims in safety cases, which help to measure claim satisfaction and detect incorrectness in safety cases (Johansson and Koopman, 2022)$^I$. SPIs also aid in the preparation of safety cases for future versions by continuously learning from the current version to be used in the next version of the safety case (Johansson and Koopman, 2022)$^I$.

*Sol7.7:* AI-based development assistance for tasks such as safety assessment and automated safety case construction can leverage Natural Language Processing (NLP) techniques, including LLMs (Perez-Cerrolaza et al., 2024)$^I$, (Sivakumar, 2024). For instance, in Sivakumar (2024) and Sivakumar et al. (2024), GPT-4 is used to generate safety cases in Goal Structuring Notation (GSN) format for an ML-based function and being compared with human generated safety case. According to the authors, "in our rigorous evaluation, GPT-4 has displayed a remarkable level of competence, achieving an excellent grade of A in its ability to work with GSN (Sivakumar et al., 2024)." However, the author also highlights the importance of human oversight and review (Sivakumar et al., 2024).

*Sol7.8:* The contract-based design approach formalises the interaction between different components and clarifies hidden assumptions by defining precise contracts (Benveniste et al., 2018; Demi et al., 2024). This approach improves the system design process and enhances the distribution of responsibilities across stakeholders. Safety and Security (SASE) contracts concept is an adaptation of contract-based design for ensuring safety and security of the system (Agirre et al., 2020)$^I$. One improvement in SASE contracts is their dynamic and adaptable nature, allowing them to accommodate modifications in components' interactions, making them suitable for DevOps. Combination of component-based designs and other approaches such as formal safety contracts can facilitate automation in safety case maintenance. However, since formal formulation is difficult, even semi-formal approaches offer benefits (Warg et al., 2019)$^I$. Moreover, the combination of modularisation and safety contracts allows for the item level safety case to only contain HARA and its validation, with a link to lower level safety cases for each module. This enables component reuse and use of modules in multiple products, as long as the contracts remain intact. Changes within the component that do not affect the safety contract will not impact an item's safety (Warg et al., 2019)$^I$. Reusable patterns for arguments and contracts foster the understandability of the safety argument (Warg et al., 2019)$^I$. According to Bate and Kelly (2002), modular architecture also lead to development costs reduction and improve maintainability. It improves verification and validation by providing measurable criteria for system behaviour that can be verified during both the development and operation phases (Hake et al., 2024)$^I$.

*Discussion:* Since the safety case is the center of all safety activities, it can be considered as the most challenging aspect for DevOps. This is why it has been the focus of several publications as summarised before. Continuous Safety Case Development (Sol7.1) and Dynamic Safety Case (Sol7.2) emphasise that it is a living work product and not a one-time activity. Fragmentation of safety case (Sol7.3) uses the divide-and-conquer approach to address system complexity and is an enabler for solutions such as SEooC (Sol7.4) and Proven in use (Sol7.5). Confidence estimation (Sol7.6) estimates uncertainty to avoid conservative restriction of ODD or specific solutions. Although confidence estimation is beneficiary, it is challenging to calculate it especially in an AD's dynamic environment.

*Industrial Context:* VSSA reports are a publicly available snapshot of the safety argumentation of AD developers, which are discussed in previous challenges. As reported, companies mostly focus and report on verification and validation, while there is little coverage of design phase activities such as requirements engineering and safety analysis. According to Aurora's Safety Case Framework (Aurora, 2025b,a), which is presented in GSN, their framework continue to evolve through their learnings from testing and expanding their operations. Analyse, build, simulate and drive is presented as a loop leading to continuous improvement of the safety (GM, 2025). Incorporating learning and safety data into next versions of software would lead to safer designs (GM, 2025). According to IKE's safety report (IKE, 2025), one of the biggest challenges of AD developers is ensuring safety during technology development due to rapid changes in hardware and software. Moreover, the complexity of system and its software which "represent years of diligent engineering and safety analysis, and in combination present even more complexity" (IKE, 2025).

### CH8: Certification and assessment

Certification of continuous updates for vehicle software is challenging due to involvement of external stakeholders such as certification parties (Kugele and Broy, 2022). Traditional certification and compliance models face challenges in managing the dynamic nature of current software update practices in modern vehicles, not only for parts containing new software but also for previously certified functions that may be affected (Santilli et al., 2024)$^I$. Normally, confirmation measures such as assessments are conducted when all activities are completed or near to the end, and this leads to long waiting time for review and feedback, which can delay updates. Moreover, changes in the software sometimes lead to the need for re-certification (Nouri et al., 2022)$^I$.

*Sol8.1:* Warg et al. propose that a combination of assessment and confirmation review with development activities can be used within the continuous development pipeline (Warg et al., 2019)$^I$. This will also push organisations towards continuous execution of safety activities as suggested in some of the previously mentioned solutions.

*Discussion:* Confirmation review, assessment, and certification can be partially automated as there are repetitive tasks such as checking for the existence of evidence and test coverage. It is feasible since there are pre-existing checklists with defined criteria that can be implemented and checked automatically. *Sol7.7* (LLM), *Sol7.8* (contract-based), and *Sol9.1* (automation) present techniques for this idea, which are discussed in detail.

### CH9: Change impact analysis and tailoring

The safety of a system is a property that results from its components and interactions, therefore, in each feature release, the entire system, including the hardware, must be analysed for the impact of changes, even if the change is only on the software. Incorporating this into rapid iteration cycles may not be seen feasible without compromising either the rapidity or the completeness of the required tasks. The latter case is the reason why the safety community does not fully welcome DevOps (Munk and Schweizer, 2022)$^I$. For instance, if a new feature is added or changes are made to any existing feature, it is necessary to analyse the impact on hazard analysis and risk assessment for both ISO 26262 and ISO 21448 (Munk and Schweizer, 2022)$^I$. If a new hazard is identified, at least one safety goal or validation target and acceptance criterion should be assigned to it. This process should be continued towards the most granular abstraction level in the functional and technical safety concept (Munk and Schweizer, 2022)$^I$, Nouri et al. (2022)$^I$.

*Sol9.1:* Incorporating automation into the continuous safety case process as presented in *Sol7.1* enables quick assessment of the impact of changes in component activities on the overall safety case (Siddique, 2020)$^I$. Automating repetitive tasks can also improve efficiency and allow safety engineers to focus on intellectual activities (Fayollas et al., 2020)$^I$. Automation of establishing traceability between artifacts and performing change impact analysis is challenging due to diversity of artifacts and their format (e..g., requirements in natural language, design artifacts in UML, and C++ codes). However, automation can be achieved using techniques such as artifacts using XML-based representation, Natural Language Processing (NLP), and matching algorithms (Rubasinghe et al., 2018). "Policy as code" (Santilli et al., 2024)$^I$ is an approach for automating compliance checks against requirements from standards and regulations throughout the DevOps cycles.

*Sol9.2:* Utilising Model-Based Systems Engineering (MBSE) and integrating safety activities therein can enhance not only efficiency but also synchronisation between development and safety activities (Munk and Schweizer, 2022)$^I$, (Czarnecki, 2019; Temmen et al., 2024)$^I$, (Tomar, 2024)$^I$. The aSET project proposes a formal safety model to improve the iterative design process using MBSE (Meyers et al., 2019).

*Sol9.3:* Traceability between artifacts, from requirements to field monitoring, is crucial for tracking how changes in one artifact impact others in each iteration (Munk and Schweizer, 2022)$^I$ (Batot et al., 2021)$^I$ (Rubasinghe et al., 2018; Wardziński and Jarzebowicz, 2023; Santilli et al., 2024)$^I$. Traceability models (Munk and Schweizer, 2022)$^I$ (Batot et al., 2021)$^I$ are enablers for ensuring consistent and comprehensive linking of safety artifacts in the development lifecycle. For instance, updating the Traceability Information Model (TIM) during each iteration, such as when there are changes in the safety goals or its attributes, can automatically identify the affected safety requirements in the Functional Safety Concept (FSC) and the Technical Safety Concept (TSC). This is then extended to include safety analysis, verification, validation, and confirmation measures such as assessments (Munk and Schweizer, 2022)$^I$. Traceability graphs can also be employed to improve visualisation and identify the impact of changes on each artifact (node) by tracking the traceability links (edges) between them (Rubasinghe et al., 2018), thereby enabling automation (*Sol9.1*). Carlan et al. proposed a semi-automated approach for analysing the impact of changes in safety arguments by enhancing the traces between argumentation elements and models of assurance artefacts (Cârlan et al., 2022)$^I$. The argument and its elements are implemented in a model-based environment, and by monitoring the changes in the specified ODD, the impact of these change on the artefacts has been identified. Blockchain-enabled frameworks such as the one proposed by Demi et al. (2024) improve traceability of changes in software artifacts even in decentralised distributed development.

*Discussion:* Although automation is proposed as a solution in *Sol9.1* and recommended in *CH8*, it is important that the outputs of automated solutions still need to be reviewed by engineers to ensure accuracy and comprehensiveness.

*Industrial Context:* According to Schnelle and Favaro (2023), the impact of software updates on the safety assessment and evaluation process in safety cases is among the open research challenges. Moreover, Schnelle and Favaro (2023) questioned the practicality of the "Change Anything, Change Everything" (CACE) approach, proposed by Sculley et al. (2015).

### CH10: Tooling

Recently, there has been a shift in using more advanced software tools in system and software development. Additionally, as mentioned earlier, most of the solutions involve automation, which in turn requires a dedicated tool chain for each solution to make it automated. For instance, continuous Integration and Deployment (CI/CD) are enablers for preserving the safety and quality of software while reducing the time between development and operations (Borg, 2021; Fuchs, 2024). The concepts of CI/CD pipelines and version control systems along with their associated tools are commonly utilised in DevOps ecosystems, including automotive industry (Munk and Schweizer, 2022)$^I$. Nevertheless, tools used in safety activities are not as advanced as the tools used for visualisation and code reviewing, which has led to safety-related activities lagging behind (Siddique, 2020)$^I$. An example of this are visualisation features, which safety-related tools often lack and are not easy to integrate with existing tools (Siddique, 2020)$^I$. Moreover, limited tool support and insufficient integration of the assurance case development process with other tools involved in development are cited as root causes of the challenges practitioners face during the development of assurance cases (Wardziński and Jarzebowicz, 2023).

*Sol10.1:* Open APIs facilitate the connection of different safety tools, which enables data-driven development for safety activities. For instance, it enables data exchange between safety analysis methods and even with simulation or real-world verification and validation tools (Siddique, 2020)$^I$. Establishing traceability between the results and assumptions in safety analysis methods, simulation, and real-world testing can enable cross-validation, avoiding mismatches and improving impact analysis. Furthermore, by allowing self-updates for safety analysis and simulation tools when real-world data changes, we can ensure their results remain accurate and up-to-date (Siddique, 2020)$^I$.

*Sol10.2:* Modernising software safety tools by adding tools to summarise the results of automatic impact analysis, e.g. by improving visualisation features (Siddique, 2020)$^I$ or implementing a safety management dashboard (Munk and Schweizer, 2022)$^I$ is a desirable solution. Another possibility for addressing *CH10* is to integrate safety activities into already existing software development tools. An example of this approach is the concept of "doc as code" (Siddique, 2020)$^I$.

*Discussion:* According to ISO 26262, "Confidence in the use of software tools" is mandatory in safety-related tool chains (Munk and Schweizer, 2022)$^I$. This process begins by identifying the Tool Confidence Level (TCL), and then, a suitable method should be chosen based on the ASIL of the function (ISO 26262:2018, 2018; Fuchs, 2024). If a tool has an impact on ASIL requirements and its failure is not detectable, then the tool validation or development process itself must comply with ISO 26262 (ISO 26262:2018, 2018; Fuchs, 2024), which can be time-consuming and that requires significant effort. Hence, it is challenging to adopt innovative and user-friendly tools for safety-related applications.

*Industrial Context:* Developing and integrating internal tools such as Requirement Tracking, Configuration Management, and Verification & Validation tools enables a robust, rapid, and iterative safety ecosystem (IKE, 2025). For instance, if the value in a requirement is updated, the software is automatically tested against this new value (IKE, 2025).

### CH11: Change in one domain affects others

As pointed out by Nouri et al. technical solutions of functional safety, SOTIF, and cyber-security are interconnected and can sometimes contradict each other (Nouri et al., 2022)$^I$. This adds another complexity dimension to the already mentioned challenges.

*Sol11.1:* The UP2DATE project (Agirre et al., 2020)$^I$ proposed safety-security co-engineering, and the SASE contracts concept as described in *Sol7.8*, which considers both safety and cyber-security simultaneously.

*Discussion:* Separation of activities in each domain might be one root cause of this challenge, which is avoided in Agirre et al. (2020)$^I$. Mapping of activities in each domain to the equivalent activity in other domains might lead to better allocation of activities in the organisation, which reduce the misalignment between domains. Integration of activities also might be a solution. For instance, integration of HARA in the context of ISO 26262 (ISO 26262:2018, 2018), Hazard Identification & Evaluation in ISO 21448 (2022), and Threat Analysis and Risk Assessment (TARA) in ISO 21434 (2021) can be a remedy for avoiding

**Table 1**
Mapping challenges (RQ1) to solutions (RQ2) proposed in literature (part 1) along with their supporting references. Papers with industrial authors are indicated by [X]$^I$, and the total number is reported in column 'I'.

| CHX & CH. Ref. | Justification of mapping | Sol. Ref. | I |
|---|---|---|---|
| CH1: Safety and software workflow separation<br>Siddique (2020)$^I$, Warg et al. (2019)$^I$ and Fayollas et al. (2020)$^I$ | Sol1.1: In addition to all independent safety experts, a "Quality Assurer" with relevant and proper safety knowledge should be added to the team to ensure the quality of safety activities. | Hanssen et al. (2018)$^I$ and Myklebust et al. (2020)$^I$ | 2 |
| CH2:Requirement updates<br>Czarnecki (2018), Bosch et al. (2018), Nouri et al. (2022)$^I$ and Habibullah et al. (2024)$^I$ | Sol2.1: Data-driven and AI-driven development besides traditional requirements engineering methods. | Bosch et al. (2018), Kugele and Broy (2022), Czarnecki (2018), Warg et al. (2019)$^I$, Habibullah et al. (2024)$^I$ and Warg et al. (2024)$^I$ | 3 |
| CH3: Safety analysis methods<br>Siddique (2020)$^I$, Thomas and Groth (2023) and Nouri et al. (2023)$^I$ | Sol3.1: Integrated safety analysis framework with live connection to system design and field data. | Siddique (2020)$^I$, Thomas and Groth (2023) and Nouri et al. (2023)$^I$ | 2 |
| | Sol3.2: Natural Language Processing (NLP) techniques, such as BERT and GPT, can be used to improve the speed and efficiency of safety analysis. | Bäumler and Prokop (2024), Zeller et al. (2024)$^I$, Nouri et al. (2024b)$^I$ and Nouri et al. (2024a)$^I$ | 3 |
| CH4: Software architecture<br>Lee et al. (2024), Amalfitano et al. (2024), Kugele and Broy (2022) and Bosch et al. (2022) | Sol4.1: Partitioning by hypervisors for freedom from interference and employing containerisation. | Kugele and Broy (2022), Kugele et al. (2018) and Lee et al. (2024) | – |
| CH5: Verification and validation<br>Siddique (2020)$^I$, Borg (2021), Stapp et al. (2024), Hoss et al. (2022), John et al. (2023) and Hartmannsgruber et al. (2024)$^I$ | Sol5.1: Simulation can cover a wider range of relevant scenarios, trace results back to requirements and enable synthetic data generation for data driven verification. | Siddique (2020)$^I$, Meyers et al. (2019), Hoss et al. (2022), Mörtstrand (2023), Hartmannsgruber et al. (2024)$^I$, Schuster et al. (2023)$^I$, Raghupatruni et al. (2021)$^I$, Habibullah et al. (2024)$^I$, Blanco et al. (2023)$^I$, Tomar (2024)$^I$ and Haas et al. (2024)$^I$ | 8 |
| | Sol5.2: Shadow mode testing can continuously verify safety and performance requirements. | Johansson and Koopman (2022)$^I$, Czarnecki (2019), Hartmannsgruber et al. (2024)$^I$, Zeller et al. (2024)$^I$, Fayollas et al. (2020)$^I$ and Blanco et al. (2023)$^I$ | 5 |
| | Sol5.3: Continuous experimentation (canary and A/B) during the operational phase. | Giaimo et al. (2019), Borg (2021) and Fayollas et al. (2020)$^I$ | 1 |
| | Sol5.4: Test automation can be achieved by employing relevant techniques for each technology, such as "Gradient-weighted Class Activation Mapping (Grad-CAM)" for ML components. | Stapp et al. (2024), Mörtstrand (2023) and Borg et al. (2021) | – |

misalignment from the very first activities in all three domains. Moreover, some solutions identified for other challenges could potentially be employed to this challenge. For example, expanding *Sol9.3* (i.e. TIM) between disciplines could help to identify the direct and indirect impact of changes across teams and disciplines (Munk and Schweizer, 2022)$^I$.

*Industrial Context:* One of the key safety design elements is the cybersecurity process of AD developers, as recommended for reporting by NHTSA (U.S. Department of Transportation NHTSA, 2017). All VSSA reports describe their cybersecurity process and their continuous monitoring and improvement efforts to safeguard their AD systems against cybersecurity threats and attacks, such as the one reported in Mars-Auto (2025).

## 6. Conclusion

Our systematic analysis and synthesis of the 319 extracted studies show that there are 11 clusters of challenges (RQ1) for using DevOps for safety-related systems such as AD. The identified challenges are mapped to the essential safety activities in the DevSafeOps loop and presented in Fig. 2. Potential solutions and relevant DevOps safety (i.e., DevSafeOps) activities are mapped to each challenge (RQ2). The findings are presented in Section 5 and Tables 1 and 2, which aggregates and represents the complex mapping between existing challenges and mitigation strategies to them.

As discussed in Section 5, we have identified gaps in each challenge when considering the proposed solutions (RQ3), such as the prerequisites needed for certain solutions and the potential to apply and study some solutions for other challenges. Additional investigation is also required to assess the coverage and effectiveness of solutions for each challenge. For instance, requirements engineering for DevOps is deemed "underexplored" by Czarnecki (Czarnecki, 2018). In addition to all aforementioned challenges, DevOps could be extended to hardware aspects of sensors, controllers and actuators (Thomas and Groth, 2023; Kugele and Broy, 2022; Nouri et al., 2022)$^I$.

As shown in Tables 1 and 2, some publications provide a high-level overview, and discuss multiple challenges and corresponding solutions, such as Czarnecki (2019), Kugele and Broy (2022), Schleiss et al. (2022), Meyers et al. (2019), Johansson and Koopman (2022)$^I$, Munk and Schweizer (2022)$^I$, Warg et al. (2019)$^I$, Siddique (2020)$^I$ and Fayollas et al. (2020)$^I$. The rest proposed one or multiple solutions for one challenge such as Bosch et al. (2018), Czarnecki (2018), Kugele et al. (2018), Giaimo et al. (2019), Borg et al. (2019)$^I$ and Raghupatruni et al. (2021)$^I$. Some solutions were discussed or investigated by multiple references, but no inconsistencies were observed. For instance, no identified reference questioned the validity or relevance of any of the identified challenges, or solutions proposed.

Certain solutions identified in this study, such as contract-based design (*Sol7.8*) and Large Language Models (*Sol7.7* and *Sol3.2*), may

**Table 2**
Mapping challenges (RQ1) to solutions (RQ2) proposed in literature (part 2) along with their supporting references. Papers with industrial authors are indicated by [X]$^I$, and the total number is reported in column 'I'.

| CHX & CH. Ref. | Justification of mapping | Sol. Ref. | I |
|---|---|---|---|
| CH6: Operation phase | Sol6.1: Dynamic and run-time risk assessment framework, along with ML techniques such as Out-of-Distribution (OOD) detection, can identify risk, weaknesses and limitations of the system during the operation phase. | Schleiss et al. (2022), Gautham et al. (2022), Furrer (2022), Brajovic et al. (2023)$^I$ and Hartmannsgruber et al. (2024)$^I$ | 2 |
| Schleiss et al. (2022), John et al. (2023) and Brajovic et al. (2023)$^I$ | Sol6.2: Operation phase monitoring by identifying relevant triggers and incident reporting | UNECE (2021)$^I$, Schleiss et al. (2022), Perez-Cerrolaza et al. (2024)$^I$ and Borg et al. (2019)$^I$ | 3 |
| CH7: Safety argumentation | Sol7.1: Progressive functional safety assessments and continuous development and release of safety case | Hoss et al. (2022), Wardziński and Jarzebowicz (2023), Schleiss et al. (2022), Warg et al. (2019)$^I$, Siddique (2020)$^I$, Myklebust et al. (2020)$^I$, Warg et al. (2024)$^I$ and Johansson and Koopman (2022)$^I$ | 5 |
| Siddique (2020)$^I$, Johansson and Sivencrona (2021)$^I$, Munk and Schweizer (2022)$^I$, Schleiss et al. (2022), Wardziński and Jarzebowicz (2023), Weiss et al. (2024)$^I$, Myklebust et al. (2020)$^I$ and Hake et al. (2024)$^I$ | Sol7.2: Dynamic safety cases and run-time monitoring to assure the system safety during run-time | Sivakumar (2024), Schleiss et al. (2022) | – |
| | Sol7.3: Fragmentation of safety cases and safety contracts to reduce re-certification efforts | Wardziński and Jarzebowicz (2023), Nouri et al. (2023)$^I$, Warg et al. (2019)$^I$ and Fayollas et al. (2020)$^I$ | 3 |
| | Sol7.4: Safety Element out of Context for component development teams in distributed development | Johansson and Sivencrona (2021)$^I$ | 1 |
| | Sol7.5: "Proven in use" argument for components not compliant with standards but incident-free | Fayollas et al. (2020)$^I$ | 1 |
| | Sol7.6: Confidence estimation to prevent excessive conservatism in safety limitations | Schleiss et al. (2022), Johansson and Koopman (2022)$^I$ and Warg et al. (2024)$^I$ | 2 |
| | Sol7.7: AI-based development assistance leverages Large Language Models for tasks such as safety assessment and automated safety case construction. | Perez-Cerrolaza et al. (2024)$^I$, Sivakumar (2024) and Sivakumar et al. (2024) | 1 |
| | Sol7.8: Formalising the interaction between different components through contract-based design. | Benveniste et al. (2018), Demi et al. (2024), Bate and Kelly (2002), Agirre et al. (2020)$^I$, Hake et al. (2024)$^I$ and Warg et al. (2019)$^I$ | 3 |
| CH8: Certification and assessment<br>Santilli et al. (2024)$^I$ and Nouri et al. (2022)$^I$ | Sol8.1: Incorporating assessment and confirmation review with development activities | Warg et al. (2019)$^I$ | 1 |
| CH9: Change impact analysis and tailoring | Sol9.1: Automation in the safety case process and assessment | Rubasinghe et al. (2018), Santilli et al. (2024)$^I$, Siddique (2020)$^I$ and Fayollas et al. (2020)$^I$ | 3 |
| Munk and Schweizer (2022)$^I$ and Nouri et al. (2022)$^I$ | Sol9.2: Model-Based System Engineering as an enabler for having machine readable design | Munk and Schweizer (2022)$^I$, Czarnecki (2019), Meyers et al. (2019), Temmen et al. (2024)$^I$ and Tomar (2024)$^I$ | 3 |
| | Sol9.3: Traceability Information Model and graph for design aspects and safety artefacts and block-chain | Munk and Schweizer (2022)$^I$, Cârlan et al. (2022)$^I$, Batot et al. (2021)$^I$, Rubasinghe et al. (2018), Demi et al. (2024), Wardziński and Jarzebowicz (2023) and Santilli et al. (2024)$^I$ | 4 |
| CH10: Tooling | Sol10.1: Open APIs to facilitate data exchange between safety analysis, simulation, and V&V | Siddique (2020)$^I$ | 1 |
| Munk and Schweizer (2022)$^I$, Borg (2021), Fuchs (2024), Wardziński and Jarzebowicz (2023) and Siddique (2020)$^I$ | Sol10.2: Modernising safety tools and integrating safety activities into software development tools | Munk and Schweizer (2022)$^I$ and Siddique (2020)$^I$ | 2 |
| CH11: Change in one domain affects others<br>Nouri et al. (2022)$^I$ | Sol11.1: safety-security co-engineering and SASE contracts concept, enable simultaneous multi-domain considerations. | Agirre et al. (2020)$^I$ | 1 |

be applicable to other challenges. However, they were not explicitly proposed by the reviewed publications. This highlights the need for further research in this area to explore potential cross-cutting solutions. Moreover, Some solutions such as Model-Based System Engineering (*Sol9.2*) (Munk and Schweizer, 2022)$^I$, (Czarnecki, 2019) are prerequisite for other solutions such as Model-based simulation

(*Sol5.1*) (Mörtstrand, 2023; Blanco et al., 2023)$^I$, (Tomar, 2024)$^I$, meaning that if the foundational solutions are not implemented, the dependent solutions cannot be applied effectively.

During our study, we observed a limited number of peer-reviewed publications that address challenges and potential solutions for ensuring safety in DevOps, particularly in the context of autonomous driving. Hence, to ensure a comprehensive review, we did not exclude relevant non-peer-reviewed papers, selecting two papers (Warg et al., 2024)$^I$ (Siddique, 2020)$^I$ that was deemed appropriate meeting the criteria from our review protocol. However, it should be noted that challenges and proposed solutions in these references may require validation, despite being valuable contributions written by a practitioner. As such, the proposed challenges and solutions represent potential topics for future studies.

Integrating new technologies into safety activities is crucial for improving safety of AD and development speed, while being careful not to over-promise. As most documents are in natural text, LLMs (*Sol7.7* and *Sol3.2*) can be employed in various aspects, including requirements engineering, safety analysis, testing, and change impact analysis (Rubasinghe et al., 2018). Moreover, due to their capability to accept inputs in the form of code, their application is not limited to tasks involving natural language. However, their capabilities, weaknesses, and limitations need to be carefully studied and paired with quality assurance to ensure the validity of the results.

Solutions such as *Sol9.1*, *Sol6.2*, and *Sol3.3* are improving the efficiency and speed by using automation. As discussed in CH10 (tooling) and highlighted by Wardziński and Jarzebowicz (2023), Sivakumar (2024), Zeller et al. (2024)$^I$ and Nouri et al. (2024b)$^I$, developers shall be in the loop to review and confirm the output of these systems. The weaknesses and limitations of such systems should be identified, and relevant avoidance or mitigation strategies should be designed before fully relying on these systems in safety-related domains such as AD.

When adopting DevOps, potential dilemmas may arise such as selecting speed at the expense of other criteria like quality, which is not viable for safety-related applications. In AD, while safety is an absolute necessity, the speed would be sacrificed due to the identified challenges. Employing identified solutions can increase the speed in DevSafeOps, satisfying both safety and rapidity concurrently.

Future work, in addition to addressing the major gaps, should explore the possibility of using the proposed solutions for other challenges. Furthermore, taking a grounded theory approach to investigate and validate the data used from grey literature in this study would be a natural next step.

## CRediT authorship contribution statement

**Ali Nouri:** Writing – review & editing, Validation, Writing – original draft, Data curation, Methodology, Conceptualization. **Beatriz Cabrero-Daniel:** Writing – original draft, Writing – review & editing, Data curation, Methodology. **Fredrik Törner:** Writing – review & editing, Validation, Writing – original draft, Supervision. **Christian Berger:** Validation, Writing – original draft, Methodology, Writing – review & editing, Supervision.

## Disclaimer

The views and opinions expressed are those of the authors and do not necessarily reflect the official policy or position of Volvo Cars.

## The use of generative AI and AI-assisted technologies

During the preparation of this work the first author used ChatGPT in order to check the grammar and spelling to improve readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

This work has been partially supported by Sweden's Innovation Agency (Vinnova, diarienummer: 2021-02585), and by the Wallenberg AI Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation, Sweden.

## Data availability

No data was used for the research described in the article.

## References

a2z, 2025. Safety report a2z smart city solution. URL: https://autoa2z.ai/Safety. (accessed 28 February 2025).

Agirre, I., Onaindia, P., Poggi, T., Yarza, I., Cazorla, F.J., Kosmidis, L., Grüttner, K., Abuteir, M., Loewe, J., Orbegozo, J.M., Botta, S., 2020. UP2DATE: Safe and secure over-the-air software updates on high-performance mixed-criticality systems. In: 2020 23rd Euromicro Conference on Digital System Design. DSD, pp. 344–351. http://dx.doi.org/10.1109/DSD51259.2020.00063.

Amalfitano, D., De Luca, M., Fasolino, A.R., Pelliccione, P., Santilli, T., 2024. Characterizing software architectural metrics for continuous compliance in the automotive domain. In: 2024 IEEE 21st International Conference on Software Architecture. ICSA, IEEE, pp. 182–193.

Apple, 2025. Our approach to automated driving system safety. URL: https://www.apple.com/ads/ADS-Safety.pdf. (accessed 28 February 2025).

Aurora, 2025a. Aurora safety report. URL: https://aurora.tech/safety. (accessed 28 February 2025).

Aurora, 2025b. Aurora's safety case framework. URL: https://safetycaseframework.aurora.tech/gsn. (accessed 28 February 2025).

AutoX, 2025. Data-driven development process, the AutoX team. URL: https://www.autox.ai/blog/20200226.html. (accessed 28 February 2025).

AVSC-D-02-2024, 2024. AVSC Best Practice for Core Automated Vehicle Safety Information. Standard, SAE.

Bate, I., Kelly, T., 2002. Architectural considerations in the certification of modular systems. In: Anderson, S., Felici, M., Bologna, S. (Eds.), Computer Safety, Reliability and Security. pp. 321–333.

Batot, E.R., Cabot, J., Gérard, S., 2021. (Not) yet another metamodel for traceability. In: 2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion. MODELS-C, pp. 787–796. http://dx.doi.org/10.1109/MODELS-C53483.2021.00125.

Bäumler, M., Prokop, G., 2024. Predicting the type of road traffic accident for test scenario generation. IEEE Access.

Benveniste, A., Caillaud, B., Nickovic, D., Passerone, R., Raclet, J.B., Reinkemeier, P., Sangiovanni-Vincentelli, A., Damm, W., Henzinger, T.A., Larsen, K.G., et al., 2018. Contracts for system design. Found. Trends® Electron. Des. Autom. 12, 124–400.

Blanco, D.F., Le Mouël, F., Lin, T., Escudié, M.P., 2023. A comprehensive survey on Software as a Service (SaaS) transformation for the automotive systems. IEEE Access.

Borg, M., 2021. The AIQ meta-testbed: Pragmatically bridging academic AI testing and industrial Q needs. In: Software Quality: Future Perspectives on Software Engineering Quality: 13th International Conference, SWQD 2021, Vienna, Austria, January 19–21, 2021, Proceedings 13. Springer, pp. 66–77.

Borg, M., Englund, C., Wnuk, K., Duran, B., Levandowski, C., Gao, S., Tan, Y., Kaijser, H., Lönn, H., Törnqvist, J., 2019. Safely entering the deep: A review of verification and validation for machine learning and a challenge elicitation in the automotive industry. J. Automot. Softw. Eng. 1, 1–19. http://dx.doi.org/10.2991/jase.d.190131.001.

Borg, M., Jabangwe, R., Åberg, S., Ekblom, A., Hedlund, L., Lidfeldt, A., 2021. Test automation with grad-cam heatmaps - a future pipe segment in mlops for vision ai? In: 2021 IEEE International Conference on Software Testing, Verification and Validation Workshops. ICSTW, pp. 175–181. http://dx.doi.org/10.1109/ICSTW52544.2021.00039.

Bosch, J., Carlson, J., Olsson, H.H., Sandahl, K., Staron, M., 2022. Accelerating Digital Transformation: 10 Years of Software Center. Springer Nature.

Bosch, J., Olsson, H.H., Crnkovic, I., 2018. It takes three to tango: Requirement, outcome/data, and AI driven development. In: Hyrynsalmi, S., Suominen, A., Jud, C., Wang, X., Bosch, J., Münch, J. (Eds.), Proceedings of the International Workshop on Software-Intensive Business: Start-Ups, Ecosystems and Platforms (SiBW 2018), Espoo, Finland, December 3 2018. CEUR-WS.org., pp. 177–192, URL: https://ceur-ws.org/Vol-2305/paper14.pdf.

Brajovic, D., Renner, N., Goebels, V.P., Wagner, P., Frész, B., Biller, M., Klaeb, M., Kutz, J., Neuhuettler, J., Huber, M.F., 2023. Model reporting for certifiable AI: A proposal from merging EU regulation into AI development. ArXiv abs/2307.11525. URL: https://api.semanticscholar.org/CorpusID:260091377.

Cârlan, C., Gauerhof, L., Gallina, B., Burton, S., 2022. Automating safety argument change impact analysis for machine learning components. In: 2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing. PRDC, pp. 43–53. http://dx.doi.org/10.1109/PRDC55274.2022.00019.

Céspedes, D., Angeleri, P., Melendez, K., Dávila, A., 2020. Software product quality in devops contexts: A systematic literature review. In: Trends and Applications in Software Engineering: Proceedings of the 8th International Conference on Software Process Improvement. CIMPS 2019, Springer, pp. 51–64.

Committee, F.S., 2022. System theoretic process analysis (STPA) recommended practices for evaluations of automotive related safety-critical systems. http://dx.doi.org/10.4271/J3187_202202.

Czarnecki, K., 2018. Requirements engineering in the age of societal-scale cyber-physical systems: The case of automated driving. In: IEEE International Requirements Engineering Conference. IEEE, pp. 3–4, URL: https://ieeexplore.ieee.org/abstract/document/8491118/.

Czarnecki, K., 2019. Software engineering for automated vehicles: Addressing the needs of cars that run on software and data. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings. ICSE-Companion, IEEE, pp. 6–8, URL: https://ieeexplore.ieee.org/abstract/document/8802777/.

DeFazio, Peter A., Larsen, Rick, 2020. Final Committee Report—The Design, Development & Certification of the Boeing 737 MAX. the US House Committee on Transportation and Infrastructure, Subcommittee on Aviation. US House Committee on Transportation and Infrastructure, Washington, DC, URL: https://democrats-transportation.house.gov/imo/media/doc/2020.09.15%20FINAL%20737%20MAX%20Report%20for%20Public%20Release.pdf. (accessed 14 April 2025).

Demi, S., Sánchez-Gordón, M., Kristiansen, M., Larrucea, X., 2024. Trustworthy and collaborative traceability management: Experts' feedback on a blockchain-enabled framework. J. Softw.: Evol. Process. (e2707).

Fayollas, C., Bonnin, H., Flebus, O., 2020. SafeOps: A concept of continuous safety. In: 2020 16th European Dependable Computing Conference. EDCC, IEEE, pp. 65–68, URL: https://ieeexplore.ieee.org/abstract/document/9237051/.

Ford, 2025. A matter of trust: Ford's approach to developing self-driving vehicles. URL: https://media.ford.com/content/dam/fordmedia/North%20America/US/2021/06/17/ford-safety-report.pdf. (accessed 28 February 2025).

Fuchs, M., 2024. Validation of Software Tools According to ISO 26262 Method 1c using Continuous Practices: Applied to the Software Product TEST-GUIDE (Ph.D. thesis). Technische Hochschule Ingolstadt.

Furrer, F.J., 2022. Safety and Security of Cyber-Physical Systems: Engineering Dependable Software using Principle-Based Development. Springer Nature.

Galbas, R., Nolte, M., Eberle, U., Hungar, H., Mosebach, H.H., Salem, N.F., Schittenhelm, H., Reich, J., Kirschbaum, T., Westhofen, L., 2024. Vv methods safety assurance position paper.

Gautham, S., Bakirtzis, G., Will, A., Jayakumar, A.V., Elks, C.R., 2022. STPA-driven multilevel runtime monitoring for in-time hazard detection. In: Trapp, M., Saglietti, F., Spisländer, M., Bitsch, F. (Eds.), Computer Safety, Reliability, and Security. Springer International Publishing, Cham, pp. 158–172.

Giaimo, F., Andrade, H., Berger, C., 2019. The automotive take on continuous experimentation: A multiple case study. In: 2019 45th Euromicro Conference on Software Engineering and Advanced Applications. SEAA, IEEE, pp. 126–130, URL: https://ieeexplore.ieee.org/abstract/document/8906771/.

GM, 2025. Self-driving safety report. URL: https://www.gm.com/stories/self-driving-cars. (accessed 28 February 2025).

Google, 2024a. MLOps: Continuous delivery and automation pipelines in machine learning. URL: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning. (accessed 01 September 2024).

Google, 2024b. What is DevOps?. URL: https://cloud.google.com/devops. (accessed 01 September 2024).

Gudla, R., Telidevulapalli, V.S., Kota, J.S., Mandha, G., et al., 2022. Review on self-driving cars using neural network architectures. World J. Adv. Res. Rev. 16, 736–746.

Haas, L., Haider, A., Jung, R., Poguntke, T., Schick, B., Schneider, S.A., Wagner, F., Zeh, T., Benedikt, M., Böde, E., et al., 2024. Controlling risk for highly automated transportation systems operating in complex open environments: A white paper of the SafeTRANS closing the gap initiative.

Habibullah, K.M., Heyn, H.M., Gay, G., Horkoff, J., Knauss, E., Borg, M., Knauss, A., Sivencrona, H., Li, P.J., 2024. Requirements and software engineering for automotive perception systems: an interview study. Requir. Eng. 1–24.

Hake, G., Reiher, D., Mentjes, J., Hahn, A., 2024. Integrating scenario-and contract-based verification for automated vessels. J. Mar. Sci. Technol. 1–21.

Hanssen, G.K., Stålhane, T., Myklebust, T., 2018. SafeScrum®-Agile Development of Safety-Critical Software. Springer.

Hartmannsgruber, A., Pinke, C., Jing, C., Pangottil, S., Grewe, R., Strauss, M., Mottok, J., Rojas, R., 2024. Improving 3d object detection for autonomous driving–a case study of data-driven development. In: International ATZ Conference. Springer, pp. 77–91.

Heyn, H.M., Knauss, E., Malleswaran, I., Dinakaran, S., 2024. An empirical investigation of challenges of specifying training data and runtime monitors for critical software with machine learning and their relation to architectural decisions. Requir. Eng. 29, 97–117.

Hoss, M., Scholtes, M., Eckstein, L., 2022. A review of testing object-based environment perception for safe automated driving. Automot. Innov. 5, 223–250.

IKE, 2025. IKE safety report. URL: https://global-uploads.webflow.com/5d5c3db08993c73c35421c12/5d9e954a4a6e6c7c7b0804d4_Ike-SafetyReport-AC-03.pdf#page=8.34. (accessed 28 February 2025).

International Electrotechnical Commission (IEC), 2010. IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems. Available online: https://webstore.iec.ch/publication/22273.

ISO 21434, 2021. ISO/SAE 21434:2021, Road Vehicles — Cybersecurity Engineering. Standard, International Organization for Standardization.

ISO 21448, 2022. ISO 21448, Road Vehicles — Safety of the Intended Functionality. Standard, International Organization for Standardization.

ISO 26262:2018, 2018. ISO 26262:2018 (All Parts), Road Vehicles — Functional Safety. Standard, International Organization for Standardization.

ISO/TR 4804:2020, 2020. ISO/TR 4804:2020, Road Vehicles – Safety and Cybersecurity for Automated Driving Systems – Design, Verification and Validation Methods. Standard, International Organization for Standardization.

Johansson, R., Koopman, P., 2022. Continuous learning approach to safety engineering. In: CARS - Critical Automotive Applications: Robustness & Safety. Zaragoza, Spain, URL: https://hal.science/hal-03782627.

Johansson, R., Sivencrona, H., 2021. Developing SEooC -original concepts and implications when extending to ADS. In: CARS 2021 6th International Workshop on Critical Automotive Applications: Robustness & Safety. Münich, Germany, URL: https://hal.science/hal-03366362.

John, M.M., Olsson, H.H., Bosch, J., 2023. Towards an AI-driven business development framework: A multi-case study. J. Softw.: Evol. Process. 35 (e2432).

Kitchenham, B.A., Charters, S., 2007. Guidelines for Performing Systematic Literature Reviews in Software Engineering. Technical Report EBSE 2007-001, Keele University and Durham University Joint Report, URL: https://www.elsevier.com/_data/promis_misc/525444systematicreviewsguide.pdf.

Kodiak, 2025. Kodiak safety report 2020. URL: https://kodiak.cdn.prismic.io/kodiak/1233c64e-7ecd-4af8-8bea-00c31b6ae959_Kodiak-2020-Safety-Report.pdf. (accessed 28 February 2025).

Koopman, P., 2023. Ul 4600: What to include in an autonomous vehicle safety case. Computer 56, 101–104. http://dx.doi.org/10.1109/MC.2023.3236171.

Koopman, P., 2024. Anatomy of a robotaxi crash: Lessons from the cruise pedestrian dragging mishap. arXiv preprint arXiv:2402.06046.

Koopman, P., Widen, W.H., 2023. Breaking the tyranny of net risk metrics for automated vehicle safety. Available at SSRN 4634179.

Koopman, P., Widen, W., 2024. Redefining safety for autonomous vehicles. arXiv:2404.16768.

Kott III, N.J., Wellfare, M., van Lierop, T.K., Mottern, E., 2011. Safe operations of unmanned systems for reconnaissance in complex environments army technology objective (source ato). In: Unmanned Systems Technology XIII. SPIE, pp. 11–18.

Kugele, S., Broy, M., 2022. Architecture as a backbone for safe DevOps in automotive systems. In: 2022 IEEE 25th International Conference on Intelligent Transportation Systems. ITSC, IEEE, pp. 4145–4150, URL: https://ieeexplore.ieee.org/abstract/document/9922097/.

Kugele, S., Hettler, D., Peter, J., 2018. Data-centric communication and containerization for future automotive software architectures. In: 2018 IEEE International Conference on Software Architecture. ICSA, IEEE, pp. 65–6509.

Lawyer, Q.E.T., 2024. Report to the boards of directors of cruise LLC Gm cruise holdings LLC and general motors Holdings LLC regarding the October 2 2023 accident in San Francisco.

Lee, S.W., Choi, Y.G., Jeon, J.W., 2024. Enhancing autonomous driving systems through ROS2 and AWS cloud: V2I interaction and HPC data processing. In: 2024 IEEE 33rd International Symposium on Industrial Electronics. ISIE, IEEE, pp. 1–6.

Maier, R., Schlattl, A., Guess, T., Mottok, J., 2024. Causalops—towards an industrial lifecycle for causal probabilistic graphical models. Inf. Softw. Technol. 174, 107520.

Mars-Auto, 2025. Mars auto safety report. URL: https://marsauto.com/VSSA_safety_report.pdf. (accessed 28 February 2025).

Mercedes-Benz, 2025. Introducing DRIVE PILOT: An automated driving system for the highway. URL: https://group.mercedes-benz.com/dokumente/innovation/sonstiges/2023-03-06-vssa-mercedes-benz-drive-pilot.pdf#page=42.07. (accessed 28 February 2025).

Meyers, B., Gadeyne, K., Oakes, B., Bernaerts, M., Vangheluwe, H., Denil, J., 2019. A model-driven engineering framework to support the functional safety process. In: 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion. MODELS-C, IEEE, pp. 619–623, URL: https://ieeexplore.ieee.org/abstract/document/8904799/.

Mörtstrand, M., 2023. Continuous verification of automotive software.

Munk, P., Schweizer, M., 2022. DevOps and safety? SafeOps! towards ensuring safety in feature-driven development with frequent releases. In: Trapp, M., Schoitsch, E., Guiochet, J., Bitsch, F. (Eds.), Computer Safety, Reliability, and Security. SAFECOMP 2022 Workshops. Springer International Publishing, Cham, pp. 145–157.

Myklebust, T., Stålhane, T., Hanssen, G., 2020. Agile safety case and DevOps for the automotive industry. In: 30th European Safety and Reliability Conference and the 15th Probabilistic Safety Assessment and Management Conference. URL: https://www.researchgate.net/profile/Thor-Myklebust/publication/349717601_Agile_Safety_Case_and_DevOps_for_the_Automotive_Industry/links/63b7c1eb03aad5368e6b94f0/Agile-Safety-Case-and-DevOps-for-the-Automotive-Industry.pdf.

Myklebusta, T., Stålhaneb, T., Wuc, S., Agile safety case for vehicle trial operations. Probabilistic Safety Assessment and Management PSAM 16.

Nouri, A., Berger, C., Törner, F., 2022. An industrial experience report about challenges from continuous monitoring, improvement, and deployment for autonomous driving features. In: Euromicro Conference on Software Engineering and Advanced Applications. pp. 358–365. http://dx.doi.org/10.1109/SEAA56994.2022.00063.

Nouri, A., Berger, C., Törner, F., 2023. On STPA for distributed development of safe autonomous driving: An interview study. In: Proceedings of the 49th EUROMICRO Conference on Software Engineering and Advanced Applications. SEAA, DURRES, ALBANIA.

Nouri, A., Cabrero-Daniel, B., Torner, F., Sivencrona, H., Berger, C., 2024a. Welcome your new AI teammate: On safety analysis by leashing large language models. In: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI. Association for Computing Machinery, New York, NY, USA, pp. 172–177. http://dx.doi.org/10.1145/3644815.3644953.

Nouri, A., Cabrero-Daniel, B., Törner, F., Sivencrona, H., Berger, C., 2024b. Engineering safety requirements for autonomous driving with large language models. URL: https://arxiv.org/abs/2403.16289, arXiv:2403.16289.

NVIDIA, 2025. NVIDIA self-driving safety report. URL: https://www.nvidia.com/en-us/self-driving-cars/. (accessed 28 February 2025).

Perez-Cerrolaza, J., Abella, J., Borg, M., Donzella, C., Cerquides, J., Cazorla, F.J., Englund, C., Tauber, M., Nikolakopoulos, G., Flores, J.L., 2024. Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey. ACM Comput. Surv. 56, http://dx.doi.org/10.1145/3626314.

Raghupatruni, I., Karjee, S., Gupta, A., Naik, V., Huber, T., 2021. Towards Establishing Continuous-X Pipeline using Modular Software-in-the-Loop Test Environments. Technical Report, SAE Technical Paper, URL: https://www.sae.org/publications/technical-papers/content/2021-26-0412/.

Rubasinghe, I., Meedeniya, D., Perera, I., 2018. Traceability management with impact analysis in DevOps based software development. In: 2018 International Conference on Advances in Computing, Communications and Informatics. ICACCI, pp. 1956–1962. http://dx.doi.org/10.1109/ICACCI.2018.8554399.

Sánchez-Gordón, M., Colomo-Palacios, R., 2020. Security as culture: a systematic literature review of devsecops. In: Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops. pp. 266–269.

Santilli, T., Pelliccione, P., Wohlrab, R., Shahrokni, A., 2024. Continuous compliance in the automotive industry. IEEE Softw. 41, 134–142. http://dx.doi.org/10.1109/MS.2023.3342974.

Schleiss, P., Carella, F., Kurzidem, I., 2022. Towards continuous safety assurance for autonomous systems. In: 2022 6th International Conference on System Reliability and Safety. ICSRS, pp. 457–462. http://dx.doi.org/10.1109/ICSRS56243.2022.10067323.

Schnelle, S., Favaro, F.M., 2023. ADS standardization landscape: Making sense of its status and of the associated research questions. arXiv preprint arXiv:2306.17682.

Schuster, A., Hagmanns, R., Sonji, I., Löcklin, A., Petereit, J., Ebert, C., Weyrich, M., 2023. Synthetic data generation for the continuous development and testing of autonomous construction machinery. at-Autom. 71, 953–968.

Schwall, M., Daniel, T., Victor, T., Favaro, F., Hohnhold, H., 2020. Waymo public road safety performance data. arXiv preprint arXiv:2011.00038.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.F., Dennison, D., 2015. Hidden technical debt in machine learning systems. Adv. Neural Inf. Process. Syst. 28.

Siddique, U., 2020. SafetyOps. arXiv preprint arXiv:2008.04461.

Sivakumar, M., 2024. Design and automatic generation of safety cases of ml-enabled autonomous driving systems.

Sivakumar, M., Belle, A.B., Shan, J., Shahandashti, K.K., 2024. Prompting GPT–4 to support automatic safety case generation. Expert Syst. Appl. 255, 124653.

Stapp, L., Roman, A., Pilaeten, M., 2024. ISTQB® Certified Tester Foundation Level. Springer.

Stettinger, G., Weissensteiner, P., Khastgir, S., 2024. Trustworthiness assurance assessment for high-risk ai-based systems. IEEE Access.

Temmen, T., Meyer, M., Wachtmeister, L., Zabihi, M., Kugler, C., Christiaens, S., Rumpe, B., Andert, J., 2024. Application of model-based systems engineering methods in virtual homologation procedures for automated driving functions. pp. 1–14. http://dx.doi.org/10.1007/978-3-658-45196-7_1.

Thomas, S., Groth, K.M., 2023. Toward a hybrid causal framework for autonomous vehicle safety analysis. Proc. Inst. Mech. Eng. Part O: J. Risk Reliab. 237, 367–388.

Tomar, G., 2024. Migration of model-based software components to service-oriented architectures. ATZelectronics Worldw. 19, 50–54.

Torc-Robotics, 2025. Torc safety report. URL: https://torc.ai/wp-content/uploads/2024/06/Torc-Robotics-VSSA.pdf. (accessed 28 February 2025).

TUSIMPLE, 2025. TUSIMPLE SAFETY report. URL: https://www.tusimple.com/wp-content/uploads/2021/03/TuSimple-Safety-Report.pdf. (accessed 28 February 2025).

UNECE, 2021. UN regulation (157) - automated lane keeping systems (ALKS). URL: https://unece.org/transport/documents/2021/03/standards/un-regulation-no-157-automated-lane-keeping-systems-alks.

U.S. Department of Transportation NHTSA, 2017. A Vision for Safety 2.0. Technical Report, U.S. Department of Transportation, National Highway Traffic Safety Administration.

Volkswagen, 2025. Volkswagen's safety self-assessment for automated mobility services. URL: https://media.vw.com/assets/documents/original/17321-VolkswagenVSSAV12023.pdf. (accessed 28 February 2025).

Waabi, 2025. Waabi's voluntary safety self-assessment. URL: https://waabi.ai/vssa/. (accessed 28 February 2025).

Wardziński, A., Jarzebowicz, A., 2023. Development of the system assurance reference model for generating modular assurance cases. In: 2023 IEEE 28th Pacific Rim International Symposium on Dependable Computing. PRDC, pp. 99–110. http://dx.doi.org/10.1109/PRDC59308.2023.00022.

Warg, F., Blom, H., Borg, J., Johansson, R., 2019. Continuous deployment for dependable systems with continuous assurance cases. In: 2019 IEEE International Symposium on Software Reliability Engineering Workshops. ISSREW, IEEE, pp. 318–325, URL: https://ieeexplore.ieee.org/abstract/document/8990195/.

Warg, F., Thorsén, A., Chen, D., Henriksson, J., Rodrigues de Campos, G., 2024. Salience4cav public report: Safety lifecycle enabling continuous deployment for connected automated vehicles.

Waymo, 2025. Waymo safety report. URL: https://waymo.com/research/waymo-safety-report/. (accessed 28 February 2025).

Webb, N., Smith, D., Ludwick, C., Victor, T., Hommes, Q., Favaro, F., Ivanov, G., Daniel, T., 2020. Waymo's safety methodologies and safety readiness determinations. arXiv preprint arXiv:2011.00054.

Weiss, G., Zeller, M., Schoenhaar, H., Drabek, C., Kreutz, A., 2024. Approach for argumenting safety on basis of an operational design domain. In: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI. pp. 184–193.

Zeller, M., Waschulzik, T., Schmid, R., Bahlmann, C., 2024. Toward a safe MLOps process for the continuous development and safety assurance of ML-based systems in the railway domain. AI Ethics 4, 123–130.

ZOOX, 2025. SAFETY innovation AT ZOOX. URL: https://www.datocms-assets.com/106048/1696537818-zoox_safety_report_volume1_2018.pdf. (accessed 28 February 2025).